



优化方法与机器学习

唐冲 教授

工程抗震研究所

建设工程学院

1. 引言 (Three key scientific tasks)
2. 神经网络 (Machine Learning)
3. Kolmogorov-Arnold Networks (KANs)
4. 基于物理信息的机器学习 (Physics-Informed Machine Learning or Scientific Machine Learning)
5. 偏差分析 (Source of Uncertainty)

1. 引言

1.1 Key scientific task 1: **Simulation**

$$b = \mathcal{F}(a)$$

- a = set of **input** conditions
- $\mathcal{F}$ = Physical model of the system (usually PDEs)
- b = **resulting** properties given $\mathcal{F}$ and a

1. 引言

1.1 Key scientific task 1: **Simulation**

$$b = \mathcal{F}(a)$$

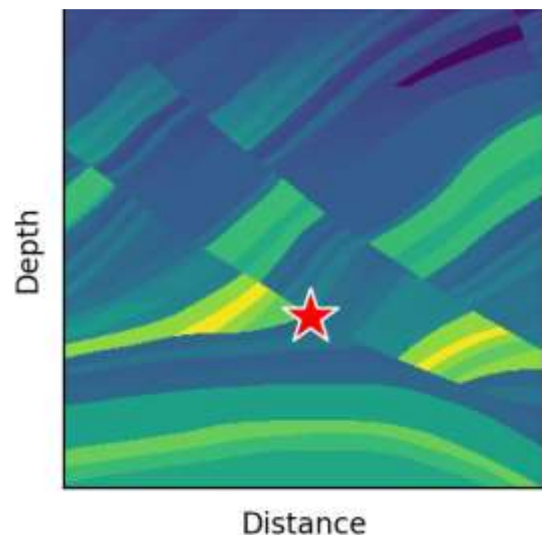
- Crucial for practically all domains of science
- Essential for understanding the behavior of complex phenomena
- Usually used as a starting point for other task (e.g., inverse/control/design problems)

1. 引言

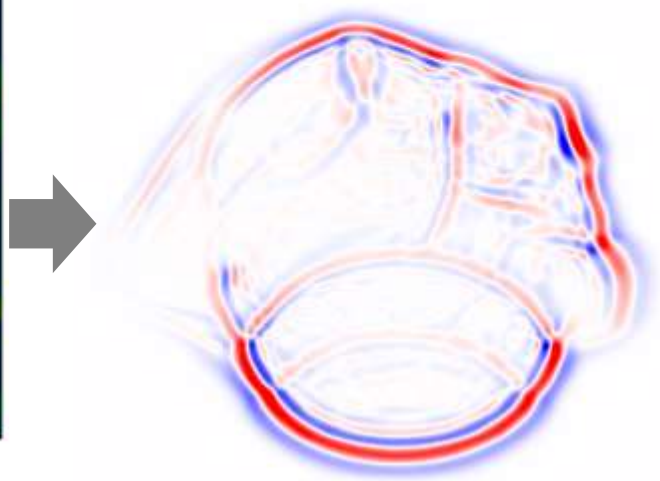
1.1 Key scientific task 1: Simulation

$$b = \mathcal{F}(a)$$

a = velocity model, $c(x, y)$, source location, s



b = wavefield, $u(x, t)$



$\mathcal{F}$ = A method for solving the wave equation (e.g. finite difference simulation)

$$\nabla^2 u - \frac{1}{c(x)^2} \frac{\partial^2 u}{\partial t^2} = \delta(x = s, t = 0)$$

$$u(x, t = 0) = 0$$

$$u'(x, t = 0) = 0$$

1. 引言

1.1 Key scientific task 1: **Simulation**

$$b = \mathcal{F}(a)$$

- Usually, analytical solutions do not exist, and we must resort to **numerical** modelling
- Many approaches exist, the most appropriate choice highly depends on the specific problem and scientific domain
- Popular methods (for systems modeled by PDEs) (e.g., finite difference methods, finite element methods, finite volume methods, spectral methods, domain decomposition, mesh-free methods)

1. 引言

1.1 Key scientific task 1: **Simulation**

$$b = \mathcal{F}(a)$$

- Typically, **computational cost** is the main challenge
- Simulation can require elaborate parallel software implementations (especially for multi-scale, multi-physics systems)
- Significant **human effort** is often required (e.g., in defining high-quality meshes for finite element simulations)

1. 引言

1.1 Key scientific task 2: Inverse problems

$$b = \mathcal{F}(a)$$

- a = set of **input** conditions
- $\mathcal{F}$ = Physical model of the system (usually PDEs)
- b = resulting properties given F and a

1. 引言

1.1 Key scientific task 2: Inverse problems

$$b = \mathcal{F}(a)$$

- Inverse problems are **pervasive** across all domains of science and solving them is essential for many real-world tasks
- Example inverse problems:
 - ① Seismic imaging (地震成像)
 - ② Magnetic resonance imaging (磁共振成像)
 - ③ Image denoising (图像降噪)
 - ④ Design problems (设计问题)
 - ⑤

1. 引言

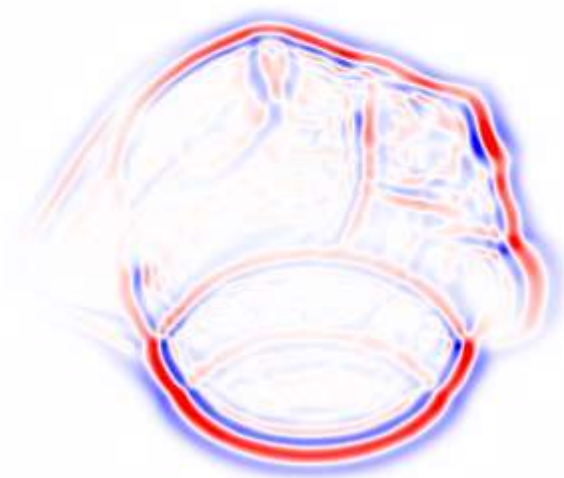
1.1 Key scientific task 2: Inverse problems

$$b = \mathcal{F}(a)$$

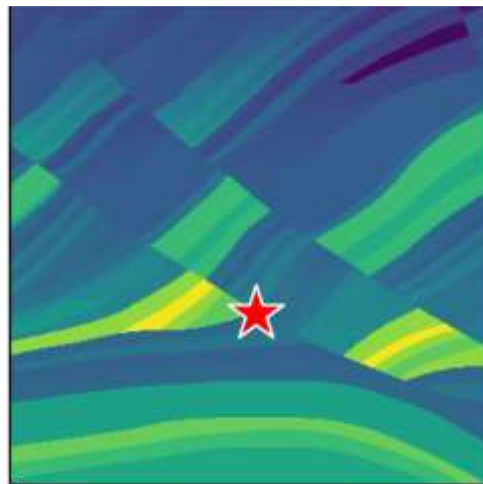
b =wavefield, $u(x, t)$

a =velocity model, source location, s

$\mathcal{F}$ = A method for solving the wave equation (e.g. finite difference simulation)



Depth



Distance

1. 引言

1.1 Key scientific task 2: Inverse problems

$$b = \mathcal{F}(a)$$

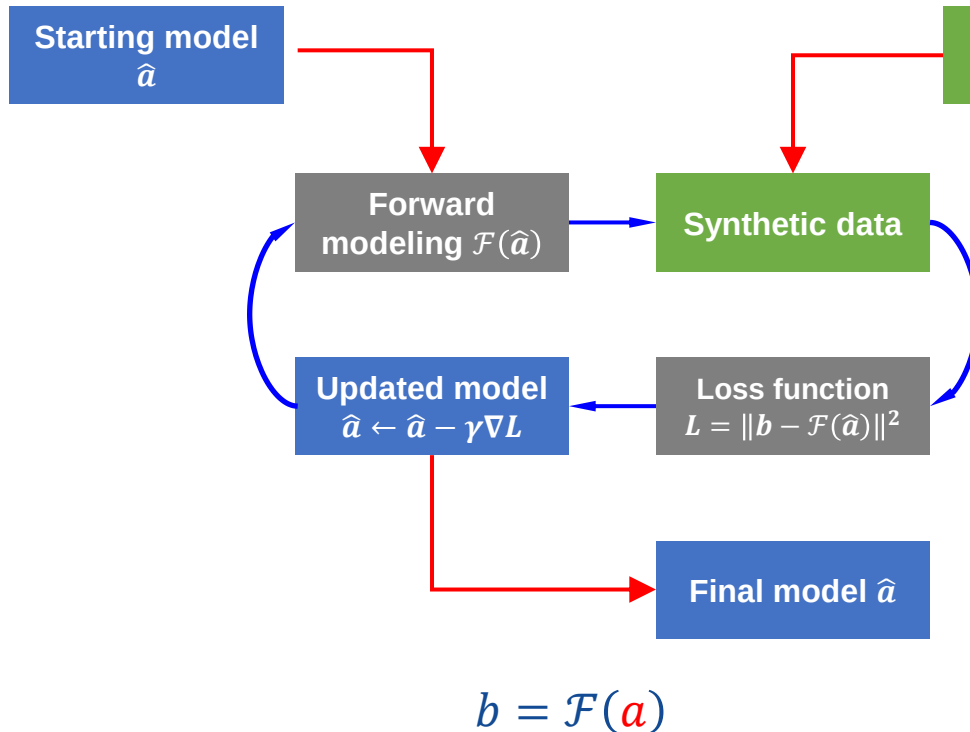
- Fundamentally, inverse problems are **search** problems
- It is often useful to frame them as an **optimization** problem for example:

$$\min_{\hat{a}} \|b - \mathcal{F}(\hat{a})\|^2$$

- If $\mathcal{F}$ is differentiable, one option is to solve the optimization problem by the Gradient-based methods (e.g., **gradient descent**)
- Otherwise, gradient-free methods can be used (e.g., evolutionary algorithms, Bayesian optimization)

1. 引言

1.1 Key scientific task 2: Inverse problems



➤ **Objective function:**

$$\min_{\hat{a}} \|b - \mathcal{F}(\hat{a})\|^2$$

➤ **Loss function**

$$L = \|b - \mathcal{F}(\hat{a})\|^2$$

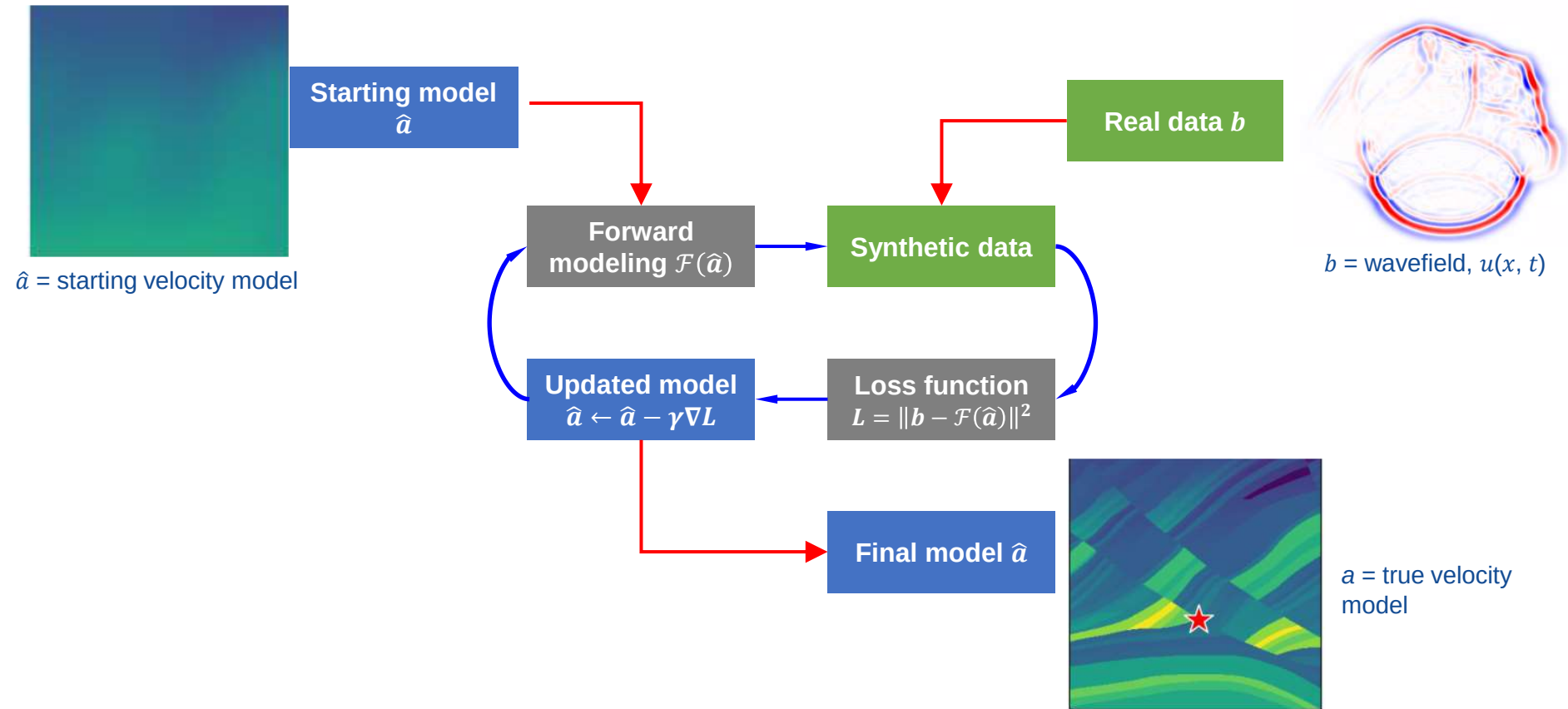
➤ **Gradient descent**

$$\hat{a} \leftarrow \hat{a} - \gamma \nabla L$$

– Requires F to be differentiable

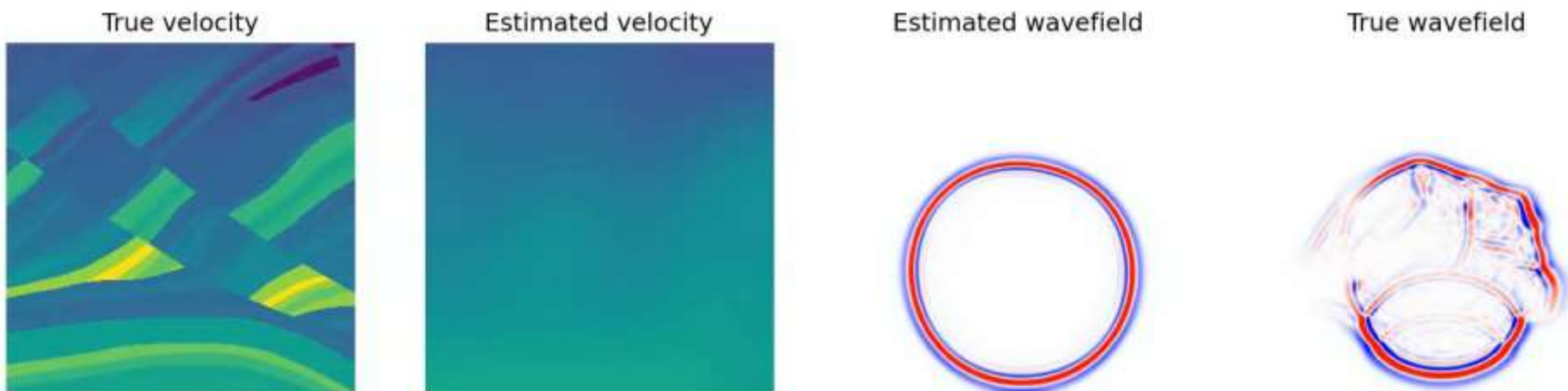
1. 引言

1.1 Key scientific task 2: Inverse problems



1. 引言

1.1 Key scientific task 2: Inverse problems



$$\min_{\hat{a}} \|b - \mathcal{F}(\hat{a})\|^2$$

- In **geophysics**, this inverse problem is known as **full waveform inversion**.

1. 引言

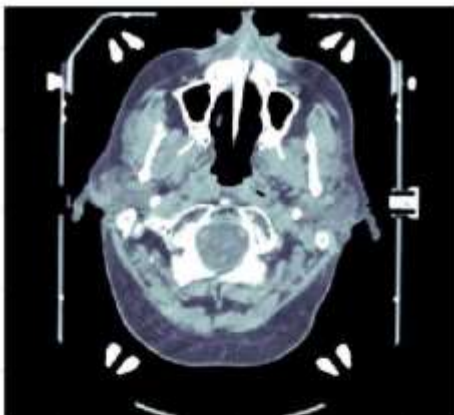
1.1 Key scientific task 2: Inverse problems

$$b = \mathcal{F}(a)$$

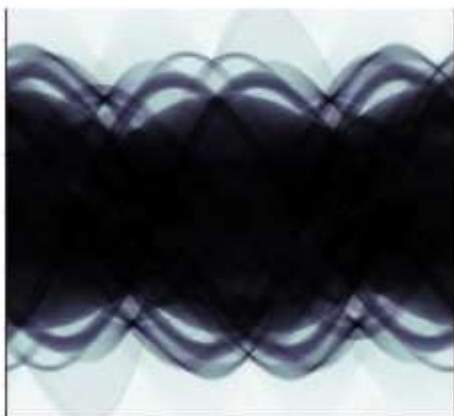
- They are usually **ill-posed** (not enough information for a unique solution)
- Observed real-world data is usually **noisy** and **sparse**
- Often require forward modelling to be carried out thousands of times – making them extremely **computationally demanding**

1. 引言

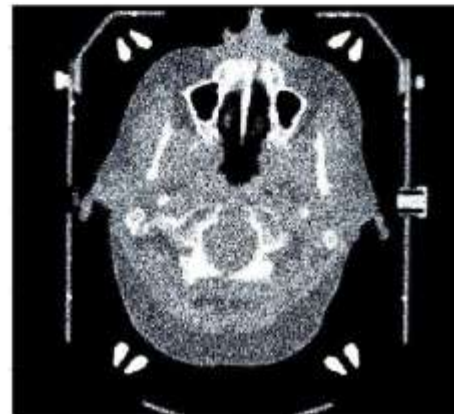
1.1 Key scientific task 2: Inverse problems



Ground truth computed tomography image



Resulting tomographic data (sinogram)



Result of inverse algorithm (filtered back-projection)

$$a(x) \quad \mathcal{F}(a)(l) = I_0 e^{-\int_l a(x) dx} \quad \hat{a}$$

1. 引言

1.1 Key scientific task 3: **Discovering unknown physics**

$$b = \mathcal{F}(a)$$

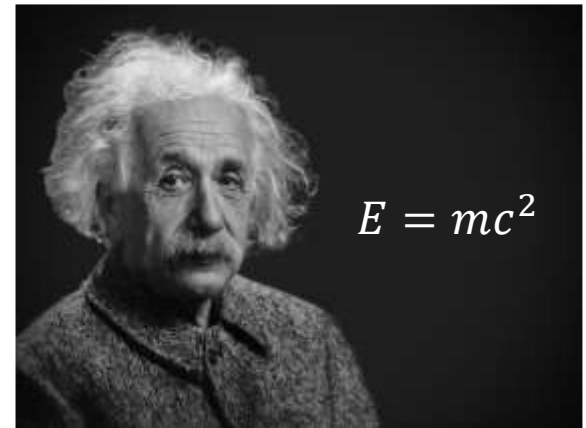
- a = set of input conditions
- $\mathcal{F}$ = **Physical model** of the system (usually PDEs)
- b = resulting properties given $\mathcal{F}$ and a

1. 引言

1.1 Key scientific task 3: **Discovering unknown physics**

$$b = \mathcal{F}(a)$$

- Historically, $\mathcal{F}$ (= laws of physics) has been found through remarkable human **intuition**
- Constant interplay between theory and experiment
- From a computational standpoint, discovering physics is like solving an inverse problem (trying to fit a model to observed data)
- But the model should be **explainable**, **generalizable** and make **novel** predictions



1. 引言

1.2 How can machine learning help?

B What are the benefits of using deep learning in science?

Deep learning, a subset of machine learning, has revolutionized various scientific fields by offering numerous benefits. Some of the benefits of using deep learning in science include:

1. Improved accuracy: Deep learning algorithms can learn patterns in large amounts of data and make highly accurate predictions. This is particularly useful in fields like medical imaging where human-level accuracy is required to detect diseases and anomalies.
2. Automation: Deep learning algorithms can automate tasks that were previously performed manually, freeing up scientists' time to focus on other tasks. For example, in astronomy, deep learning algorithms can be used to automatically identify objects in images, saving researchers hours of manual analysis.
3. Enhanced understanding: By revealing patterns and relationships in data, deep learning algorithms can help scientists gain new insights into their fields. This is particularly useful in areas such as genomics where large amounts of data need to be analyzed to uncover important biological insights.
4. New applications: Deep learning can be applied to new areas that were previously difficult to tackle using traditional machine learning methods. For example, it has been used in chemistry to predict the properties of new chemical compounds and accelerate drug discovery.
5. Increased efficiency: By reducing the time required to analyze data and perform experiments, deep learning can increase the overall efficiency of scientific research. This can lead to new discoveries being made more quickly and cost-effectively.

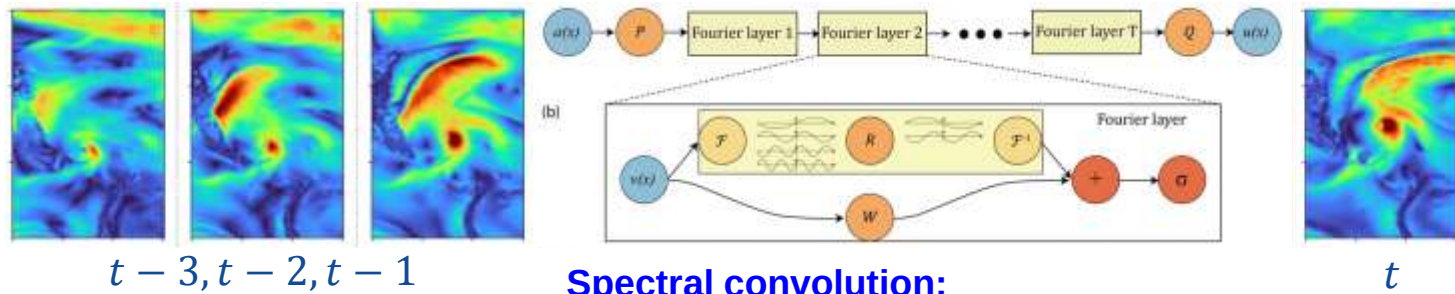
Overall, deep learning has the potential to significantly enhance the speed and accuracy of scientific research, enabling scientists to make new discoveries and progress more quickly.

- Accuracy
- Efficiency
- Automation
- Discovery

1. 引言

1.2 How can machine learning help?

① Example 1: **Simulation**



Spectral convolution:

- ① **Fourier transform:** $x = \text{FFT}(x)$
- ② **Linear Transform:** $x = x[:\text{modes}]$, $x = x * \theta$
- ③ **Inverse Fourier Transform:** $x = \text{IFFT}(x)$

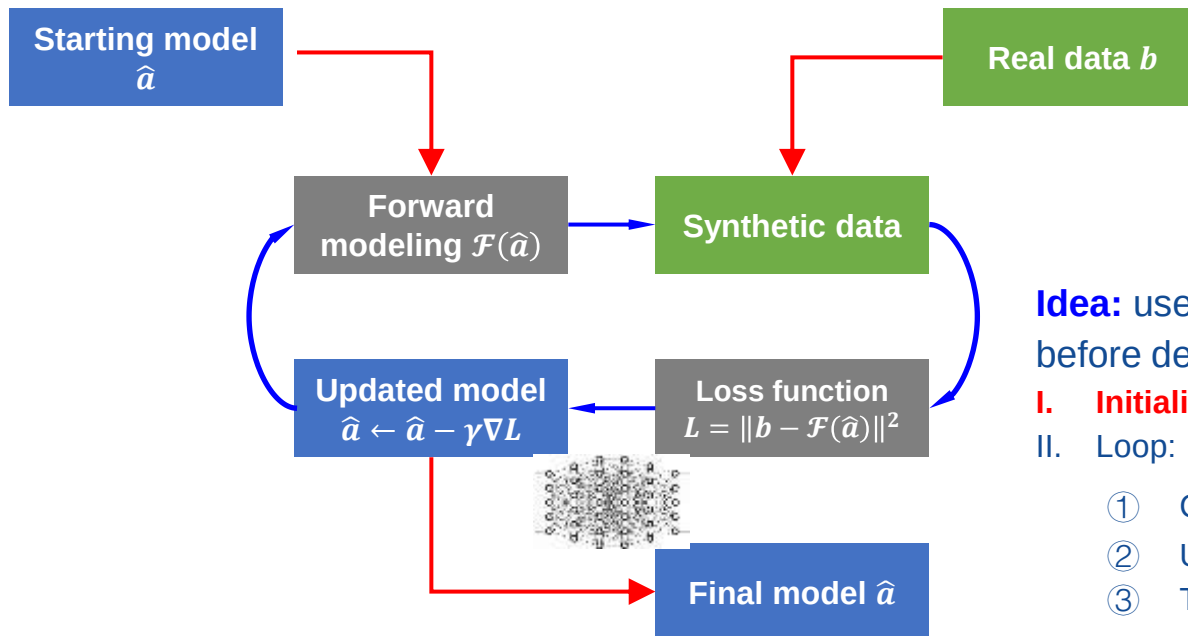
Idea: use deep neural network to predict next timestep of multiple atmospheric variables, given previous timesteps

- Use a Fourier neural operator architecture
- **4-5 orders of magnitude** faster than numerical simulation

1. 引言

1.2 How can machine learning help?

② Example 2: Inverse problems



➤ Objective function:

$$\min_{\hat{a}} \|b - \mathcal{F}(\hat{a})\|^2$$

➤ Loss function

$$L = \|b - \mathcal{F}(\hat{a})\|^2$$

➤ Gradient descent

$$\hat{a} \leftarrow \hat{a} - \gamma \nabla L$$

Idea: use a neural network to **update** gradient before descent step

I. **Initialize $\hat{a}$**

II. Loop:

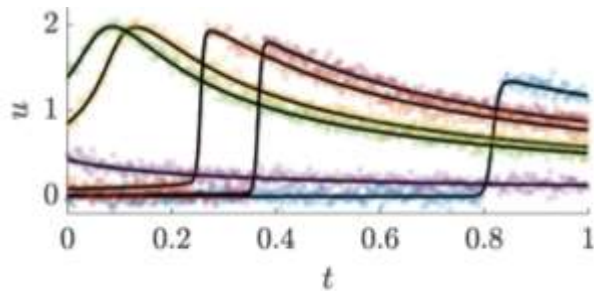
- ① Compute gradient, $g = \frac{\partial L(\hat{a})}{\partial \hat{a}}$
- ② Update gradient, $g \leftarrow \text{NN}(g, \hat{a}, b, \theta)$
- ③ Take step, $\hat{a} \leftarrow \hat{a} - \gamma g$

- Network is trained using many example inverse problems, and differentiating through entire algorithm **end-to-end**

1. 引言

1.2 How can machine learning help?

③ Example 3: Equation discovery



- **Ground truth:**

$$u_t + uu_x - 0.0032u_{xx} = 0$$

- **Discovered:**

$$u_t + 1.002uu_x - 0.0032u_{xx} = 0$$

$$u_t = \Lambda \Phi$$

$$\Lambda = (\lambda_1 \quad \lambda_2 \quad \lambda_3 \quad \dots)$$

$$\Phi = (\hat{u}_x \quad \hat{u}_{xx} \quad \hat{u}_t \quad \hat{u}_{tt} \quad \hat{u}_{xt} \quad \dots)^T$$

Idea: fit a neural network to observed data, then regress over a **library** of gradients to “discover” underlying equations

Input: $\hat{u}(x, t) = (\text{noisy})$ observational data

- ① Fit a neural network to data, $\hat{u}(x, t) = \text{NN}(x, t; \theta) \approx \tilde{u}$, using supervised learning
- ② Compute various gradients of network, for example, $\hat{u}_x, \hat{u}_{xx}, \hat{u}_t, \hat{u}_{tt}, \hat{u}_{xt}, \dots$, at many random (x, t) locations
- ③ Carry out a (sparse) **linear regression** over combinations of these gradients to “discover” underlying equation

1. 引言

1.3 Why Physics-Informed Machine Learning?

① **Example:** given training data, find a function (with flexible parameters θ) which approximates the true function

Simple polynomial regression:

$$\hat{y}(x; \theta) = \theta_4 x^3 + \theta_3 x^2 + \theta_2 x + \theta_1$$

To fit, use least-squares:

$$\theta^* = \min_{\theta} \sum_{i=1}^N [\hat{y}(x_i; \theta) - y_i]^2$$

Re-write using linear algebra:

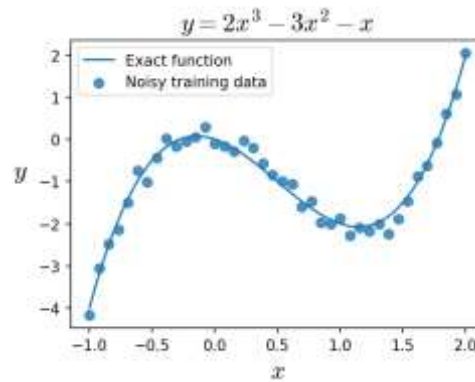
$$\begin{Bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \hat{y}_3 \\ \vdots \end{Bmatrix} = \begin{bmatrix} 1 & x_1 & x_1^2 & x_1^3 \\ 1 & x_2 & x_2^2 & x_2^3 \\ \dots & \dots & \dots & \dots \end{bmatrix} \begin{Bmatrix} \theta_1 \\ \theta_2 \\ \theta_3 \\ \theta_4 \end{Bmatrix}$$

$$\hat{\mathbf{y}} = \Phi^T \theta$$

$$\theta^* = \min_{\theta} \|\Phi^T \theta - \mathbf{y}\|^2$$

In this case, the analytical solution is:

$$\theta^* = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{y}$$



Neural Network (NN) regression:

$$\hat{y}(x; \theta) = \text{NN}(x; \theta)$$

To fit, use least-squares:

$$\theta^* = \min_{\theta} \sum_{i=1}^N [\text{NN}(x_i; \theta) - y_i]^2$$

In general, no analytical solution exists.

So, **optimization** is used.

For example, **gradient descent**:

$$\theta_j \leftarrow \theta_j - \gamma \frac{\partial \sum_{i=1}^N [\text{NN}(x_i; \theta) - y_i]^2}{\partial \theta_j}$$

Or equally

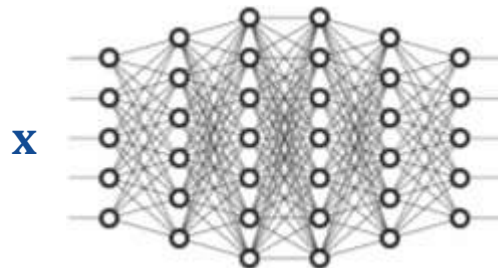
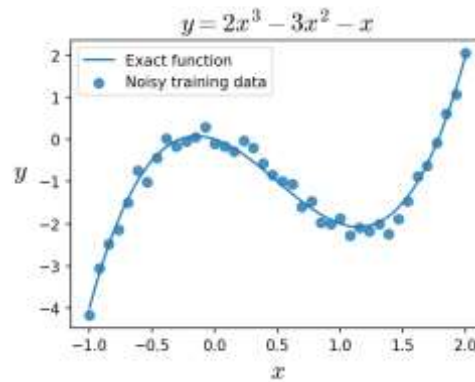
$$\theta_j \leftarrow \theta_j - \gamma \frac{\partial L(\theta)}{\partial \theta_j}$$

where γ = learning rate.

1. 引言

1.3 Why Physics-Informed Machine Learning?

- ① **Example:** given training data, find a function (with flexible parameters θ) which approximates the true function

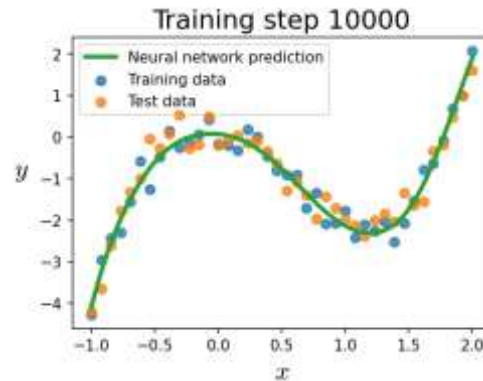


$$\hat{y} = \text{NN}(x; \theta) \approx y(x)$$

1. 引言

1.3 Why Physics-Informed Machine Learning?

- ① **Example:** given training data, find a function (with flexible parameters θ) which approximates the true function



- Train error (N):

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N |\text{NN}(x_i; \theta) - y_i|$$

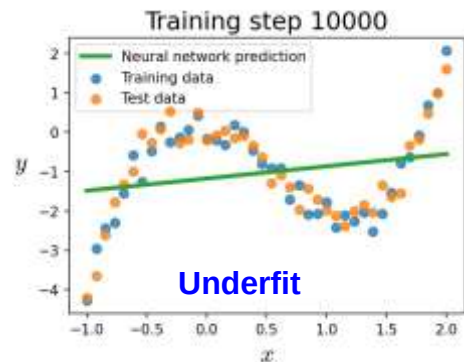
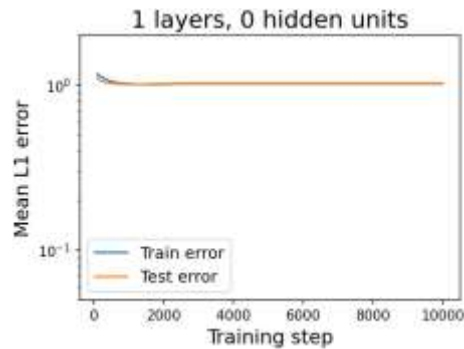
- Test error (M):

$$L(\theta) = \frac{1}{M} \sum_{i=1}^M |\text{NN}(x_i; \theta) - y_i|$$

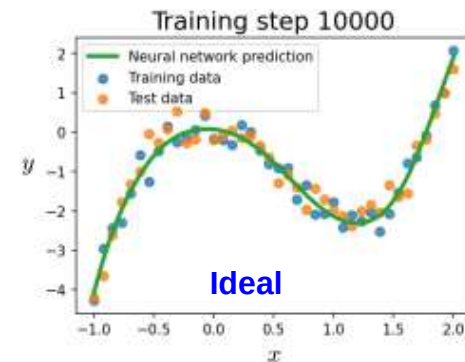
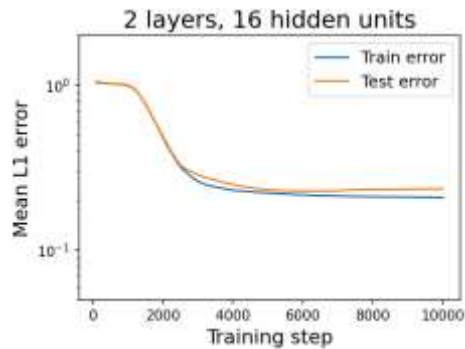
1. 引言

1.3 Why Physics-Informed Machine Learning?

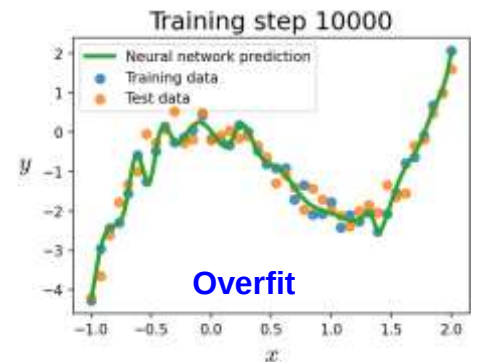
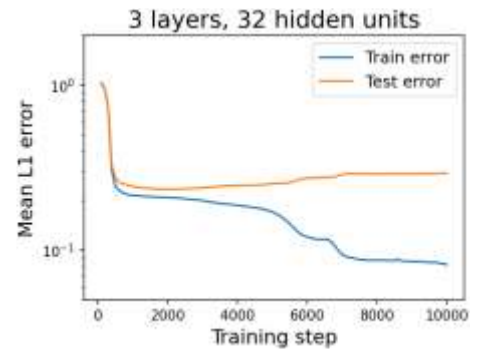
② Overfitting vs Underfitting



High bias, low variance



Low bias, low variance

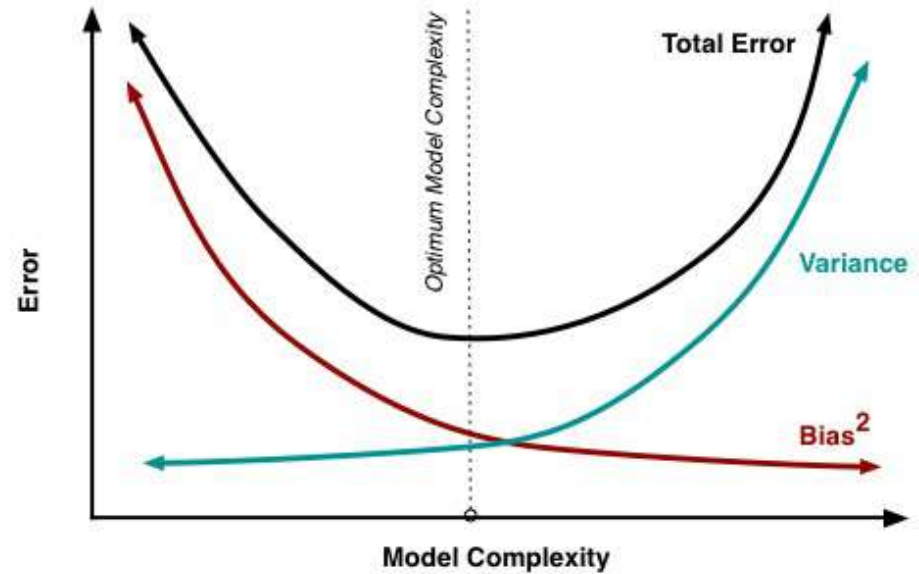
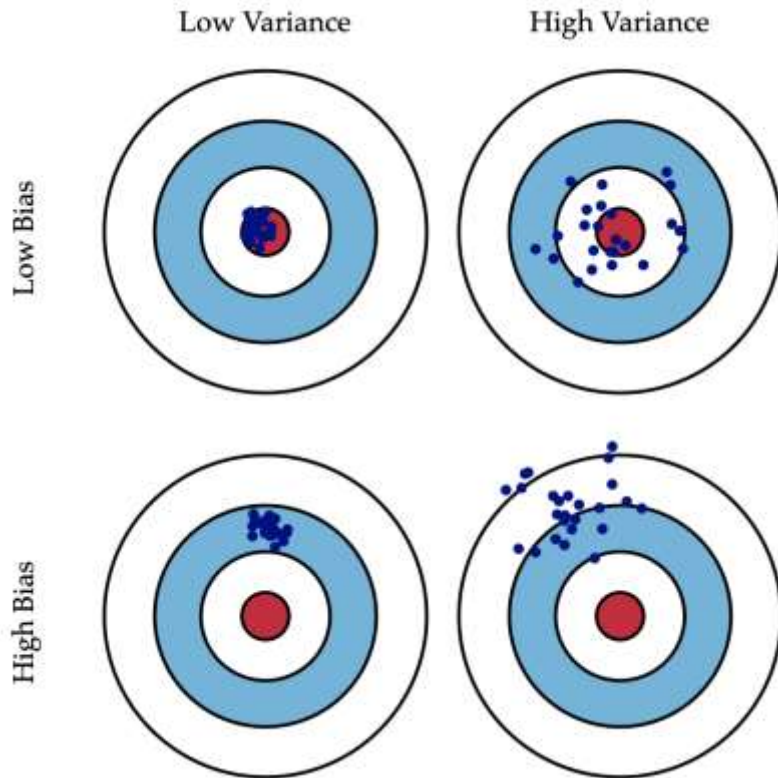


Low bias, high variance

1. 引言

1.3 Why Physics-Informed Machine Learning?

③ Overfitting vs Underfitting (Bias-variance tradeoff)



1. 引言

1.3 Why Physics-Informed Machine Learning?

③ Overfitting vs Underfitting

If you have high bias (or **underfitting**)

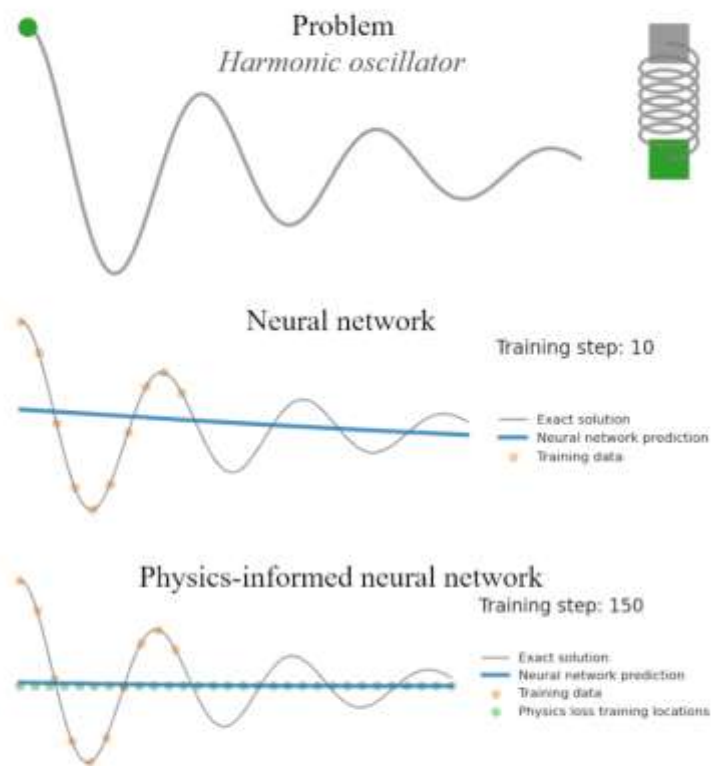
- Increase model **complexity** (e.g., number of free parameters)
- Modify model architecture

If you have high variance (or **overfitting**)

- Increase the amount of training data
- Regularize the neural network in some other way. **Regularize** = restrict the space of possible functions a neural network can learn/**prefer** certain regions of the function space/impart a **prior** on the learning algorithm (e.g., reduce model complexity, modify model architecture, increase amount of training data, weight regularization/pruning, **additional loss terms**, **dropout**, early stopping)

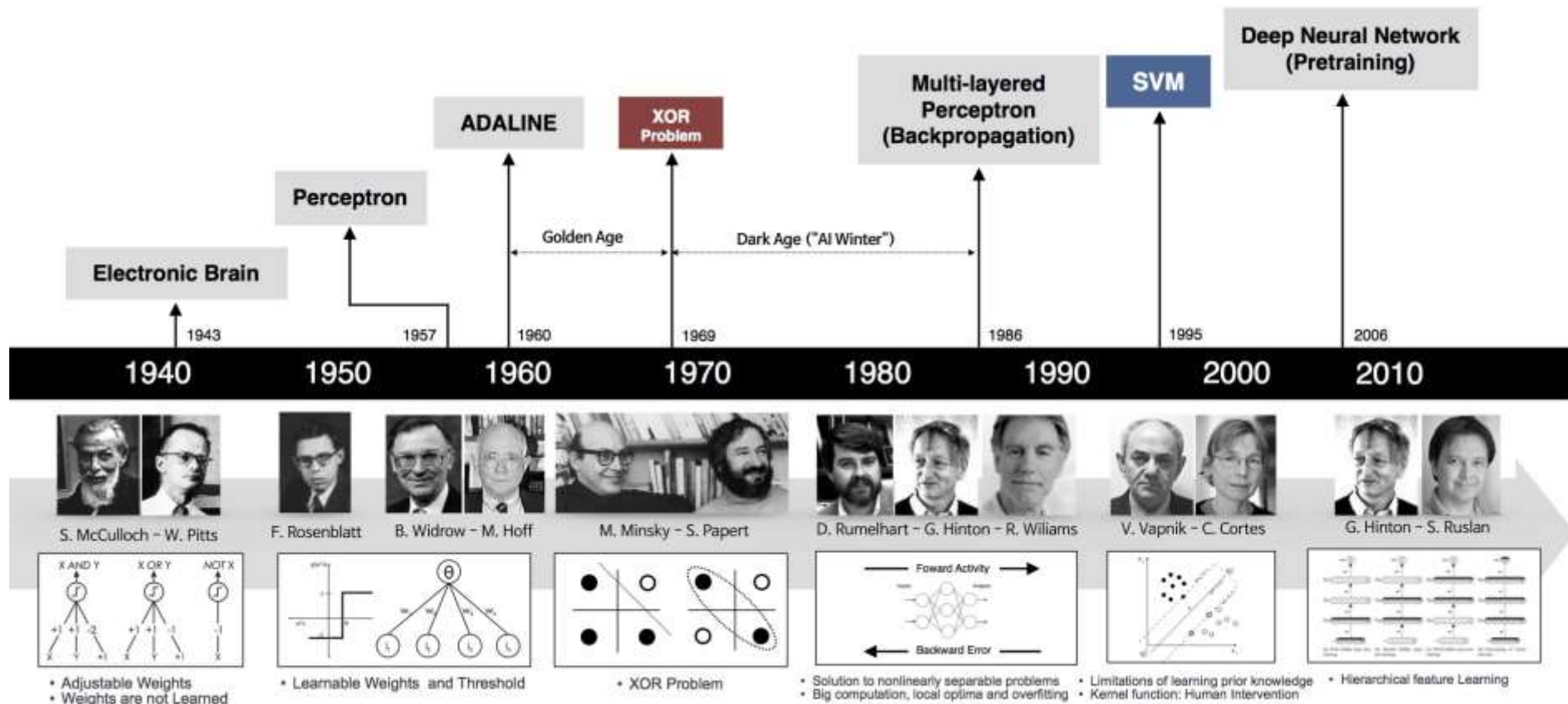
1. 引言

1.3 Why Physics-Informed Machine Learning?



2. 机器学习

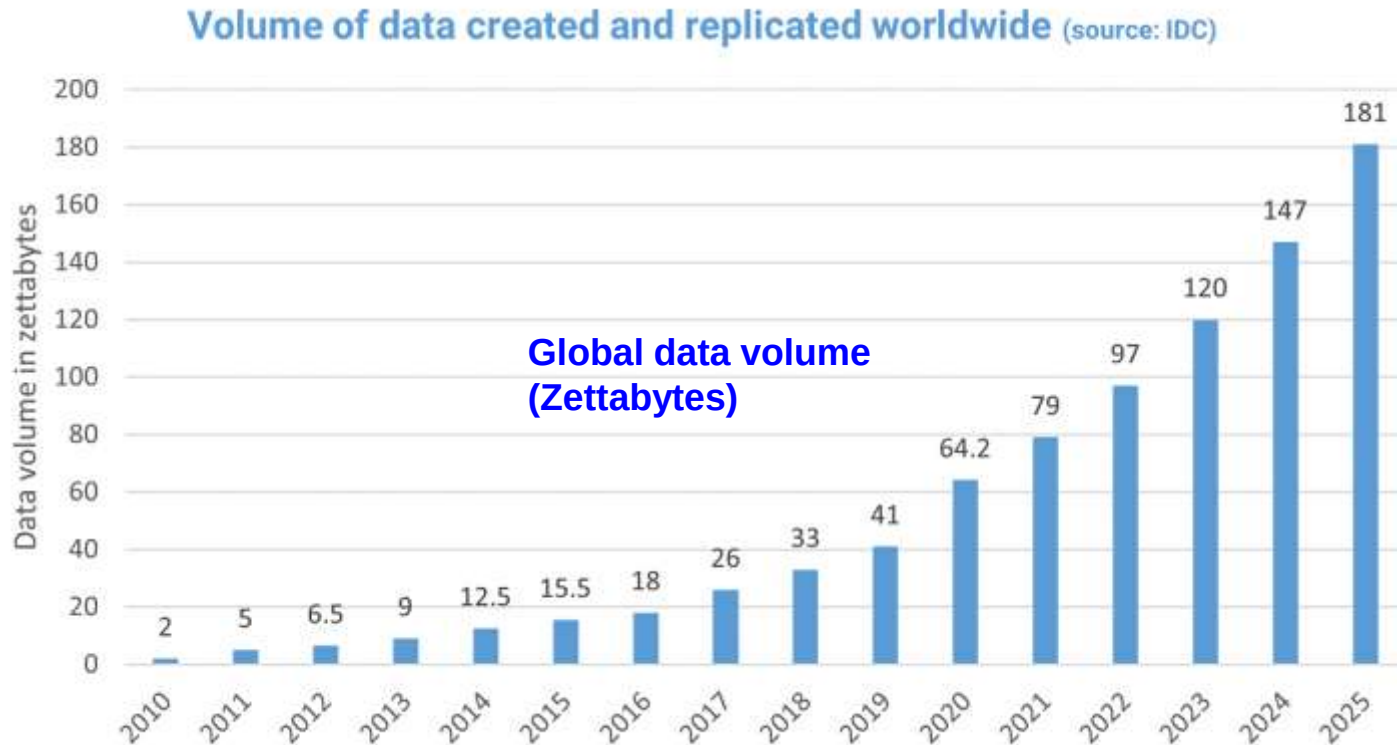
2.1 NNs date back to the 1950's – why is deep learning so popular today?



2. 机器学习

2.1 NNs date back to the 1950's – why is deep learning so popular today?

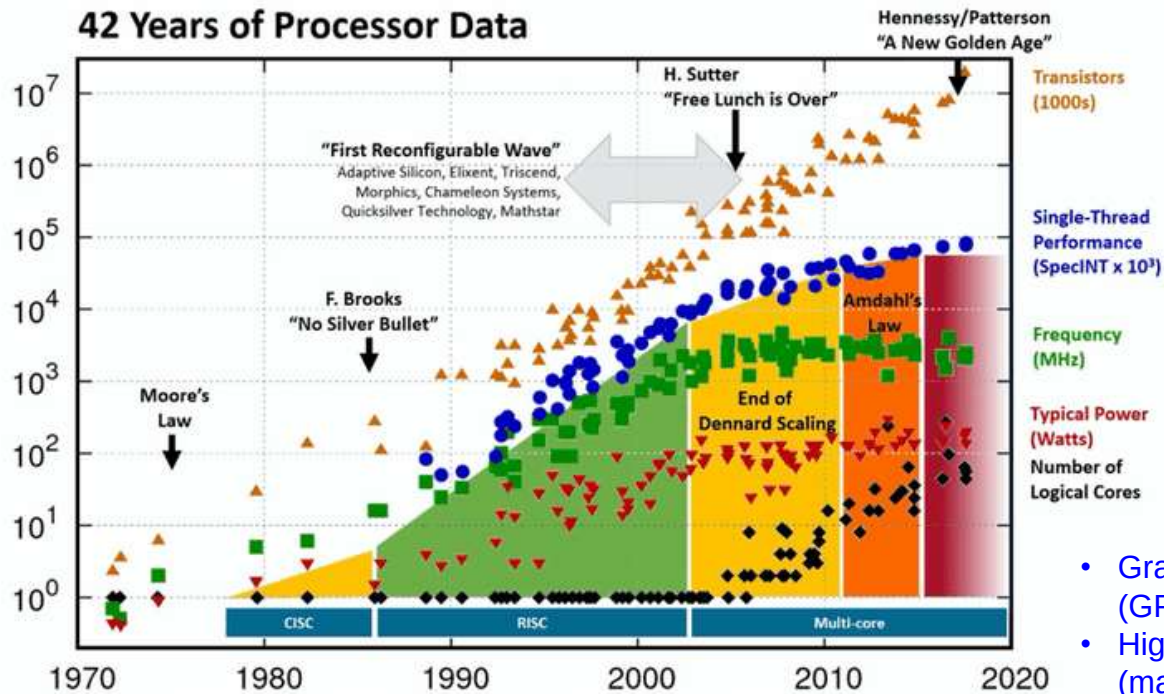
① Data availability: **Rapidly increasing amounts of data**



2. 机器学习

2.1 NNs date back to the 1950's – why is deep learning so popular today?

② Hardware: Hardware improvements



Hennessy and Patterson, Turing Lecture 2018, overlaid over "42 Years of Processors Data"
<https://www.karlsruhp.net/2018/02/42-years-of-microprocessor-trend-data/>; "First Wave" added by Les Wilson, Frank Schirmermeister
Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2017 by K. Rupp

- Graphical processing units (GPUs)
- Highly optimized for deep learning (massively parallel)

2. 机器学习

2.1 NNs date back to the 1950's – why is deep learning so popular today?

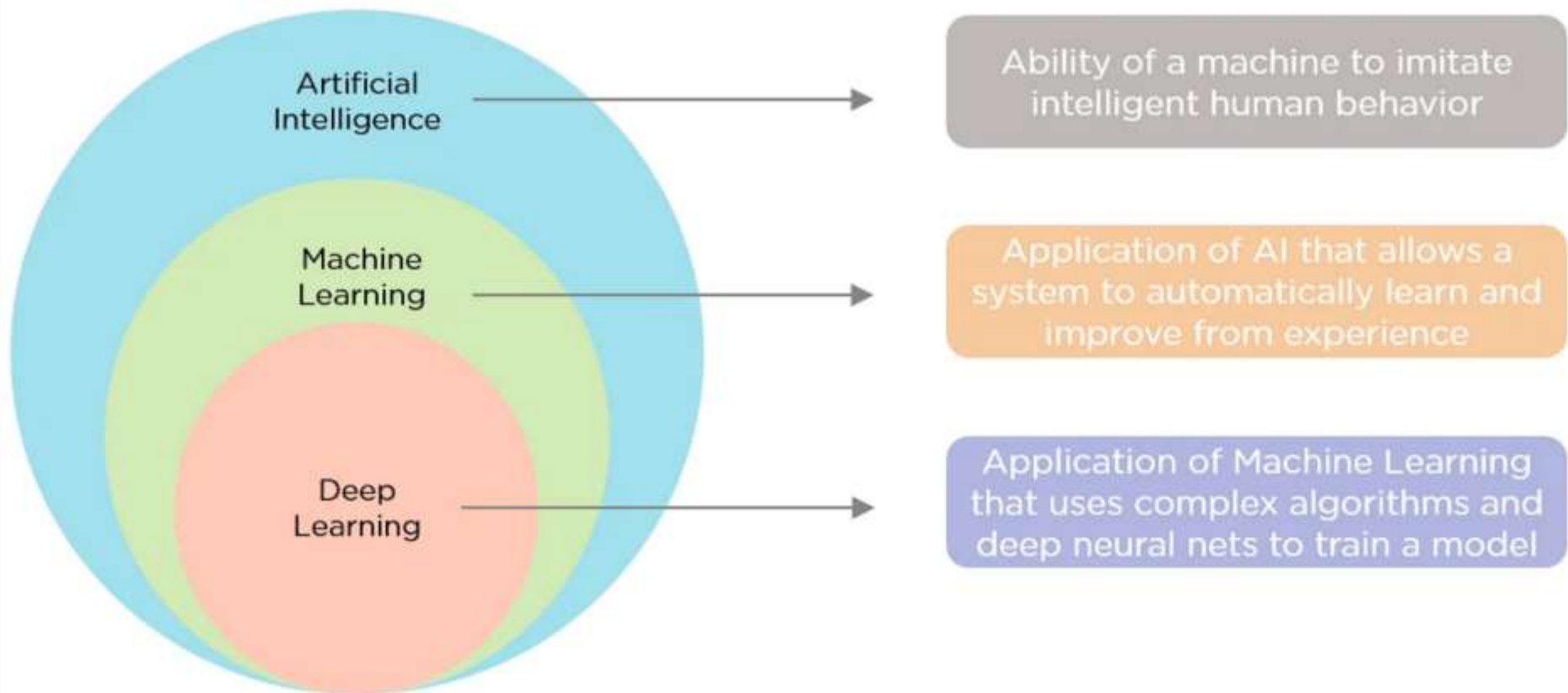
③ Software: **Software improvements**



- **Mature** deep learning frameworks
- **Better** training algorithms
- **Deeper and more sophisticated** architectures

2. 机器学习

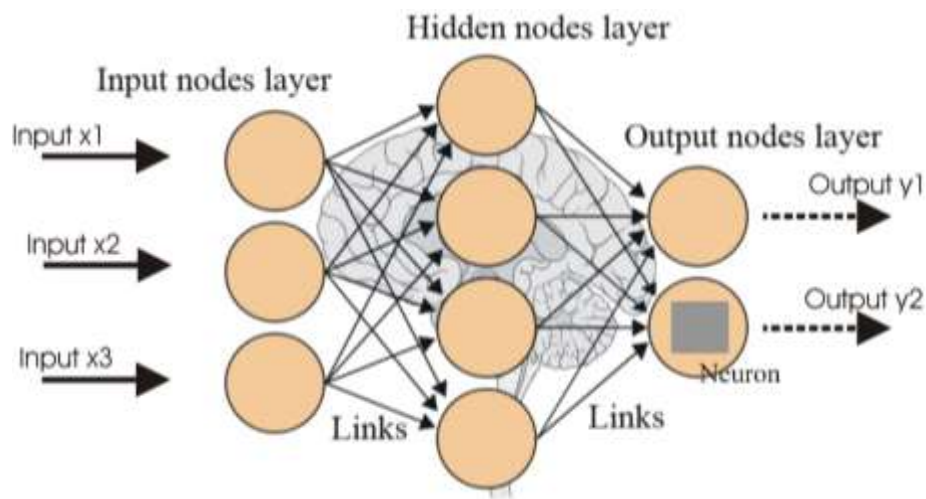
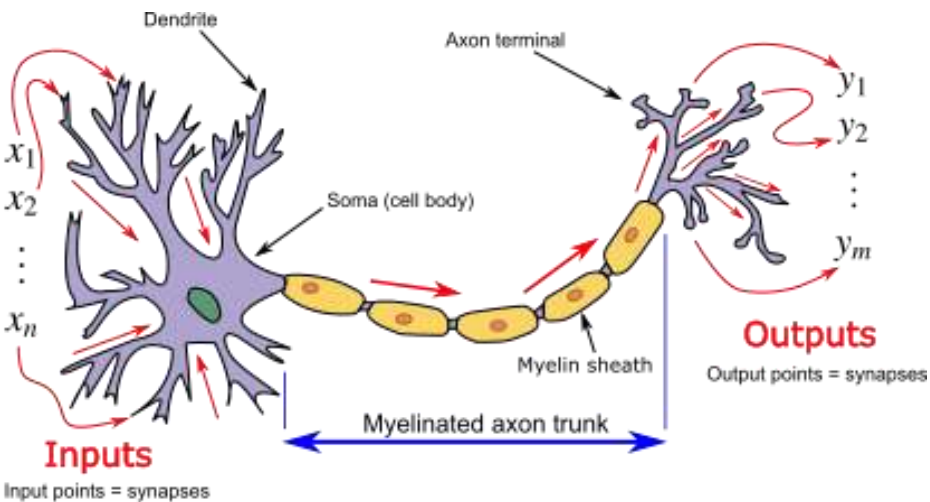
2.2 Classification



2. 机器学习

2.3 Neural Networks (NNs)

① What are the NNs?



2. 机器学习

2.3 Neural Networks (NNs)

① What are the NNs? (Universal Approximation Theorem) (全局逼近定理)

- **UAT Theorem (Cybenko 1989; Hornik 1991)**

Let $\sigma: \mathbb{R} \rightarrow \mathbb{R}$ be a *nonconstant, bounded, and continuous* function. Let $\mathbf{I}_n$ denote the n -dimensional unit hypercube $[0, 1]^n$. The space of *real-valued continuous functions on $\mathbf{I}_n$* is denoted by $C(\mathbf{I}_n)$. Then, given any $\varepsilon > 0$ and any function $f \in C(\mathbf{I}_n)$, *there exist an integer N_ε , real constants $\alpha_i, b_i \in \mathbb{R}$, and real vectors $\mathbf{w}_i \in \mathbb{R}^n$ for $i = 1, \dots, N_\varepsilon$, such that we may define*

$$F(\mathbf{x}) = \sum_{i=1}^{N_\varepsilon} \alpha_i \sigma(\mathbf{w}_i^T \mathbf{x} + b_i) \quad \mathbf{x}, \mathbf{w}_i \in \mathbb{R}^n; \alpha_i, b_i \in \mathbb{R}$$

as an approximate realization of the function f ; that is

$$|F(\mathbf{x}) - f(\mathbf{x})| < \varepsilon$$

for all $\mathbf{x} \in \mathbf{I}_n$.

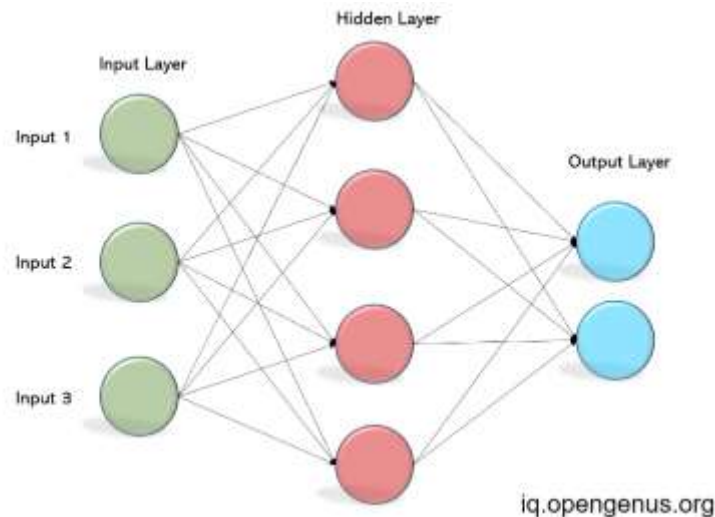
2. 机器学习

2.3 Neural Networks (NNs)

① What are the NNs? (Multilayer perceptron – MLP) (aka fully connected network)

$$\text{NN}(\mathbf{x}, \theta) = \mathbf{w}_2 \sigma(\mathbf{w}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$$

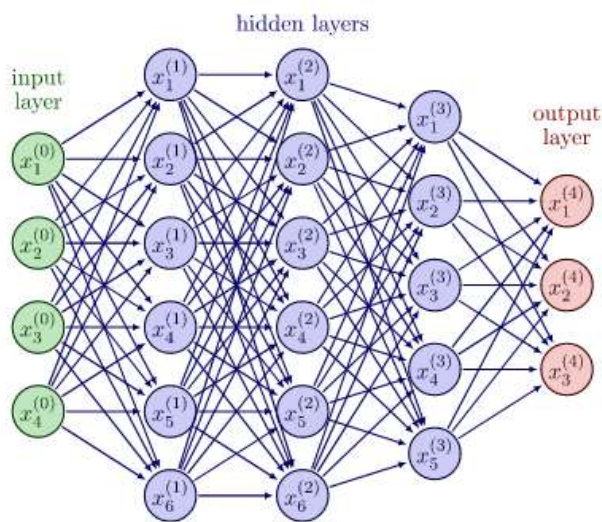
where $\mathbf{x}$ = an input vector, $\mathbf{w}_1$ and $\mathbf{w}_2$ = learnable weight matrices, $\mathbf{b}_1$ and $\mathbf{b}_2$ = learnable bias vectors, and σ = an activation function.



2. 机器学习

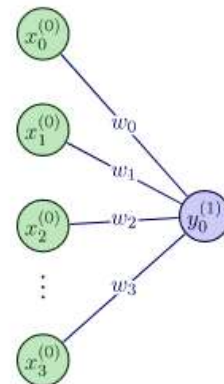
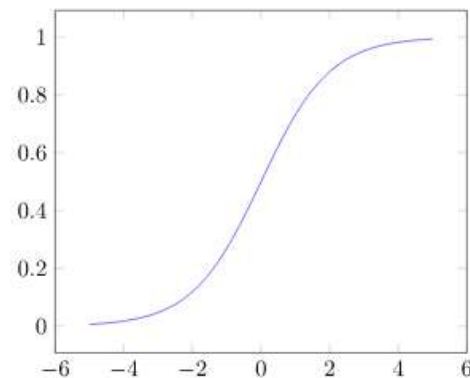
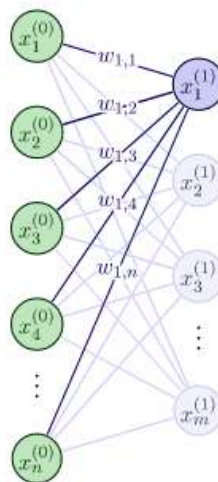
2.3 Neural Networks (NNs)

Theorem



$$W^{(l)} = \begin{pmatrix} w_{1,1}^{(l)} & w_{1,2}^{(l)} & \dots & w_{1,n}^{(l)} \\ w_{2,1}^{(l)} & w_{2,2}^{(l)} & \dots & w_{2,n}^{(l)} \\ \dots & \dots & \ddots & \dots \\ w_{m,1}^{(l)} & w_{m,2}^{(l)} & \dots & w_{m,n}^{(l)} \end{pmatrix}$$

$$\begin{aligned} \psi(x) &:= \sigma \left(\sum_{i=1}^n x_i w_i - b \right) \\ &= \sigma \left(\underbrace{w^\top \bullet x - b}_{\in \mathbb{R}} \right), \end{aligned}$$



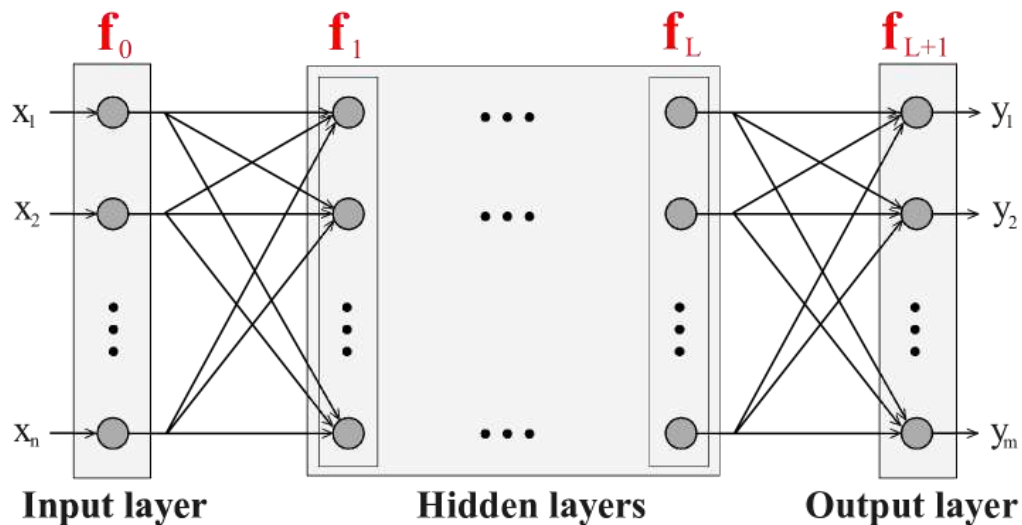
$$\int_{x \in \mathbb{R}^n} \sigma(w^\top x - b) d\mu(x) = 0 \quad \forall w \in \mathbb{R}^n, b \in \mathbb{R}$$

2. 机器学习

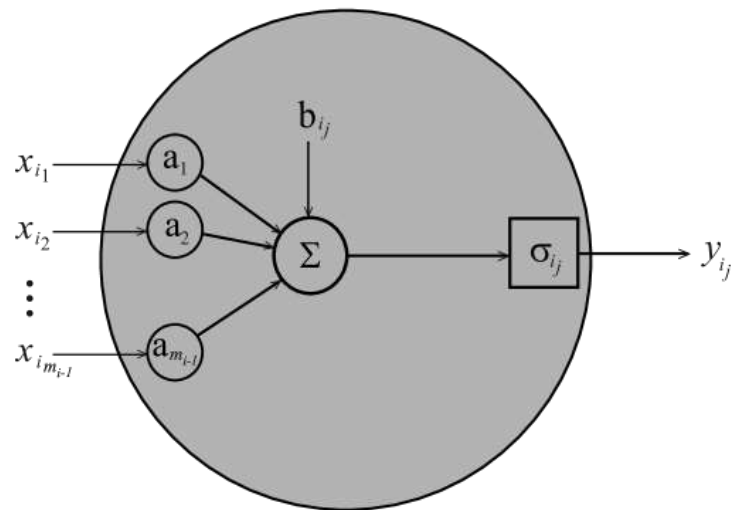
2.3 Neural Networks (NNs)

① What are the NNs? (Framework)

$$\mathbf{y} = \mathbf{f}_{\text{NN}}(\mathbf{x}) = \mathbf{f}_{L+1} \left(\mathbf{f}_L \cdots \mathbf{f}_1 (\mathbf{f}_0(\mathbf{x})) \right)$$



(a)

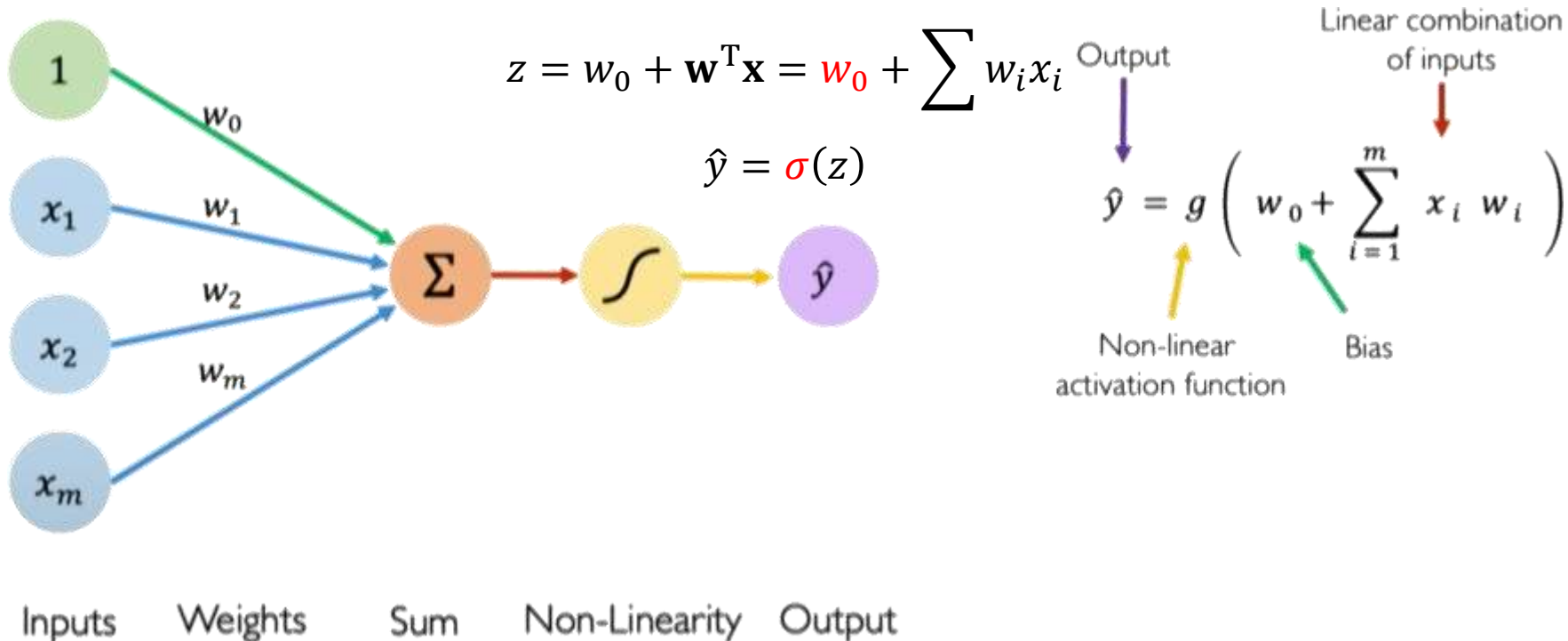


(b)

2. 机器学习

2.3 Neural Networks (NNs)

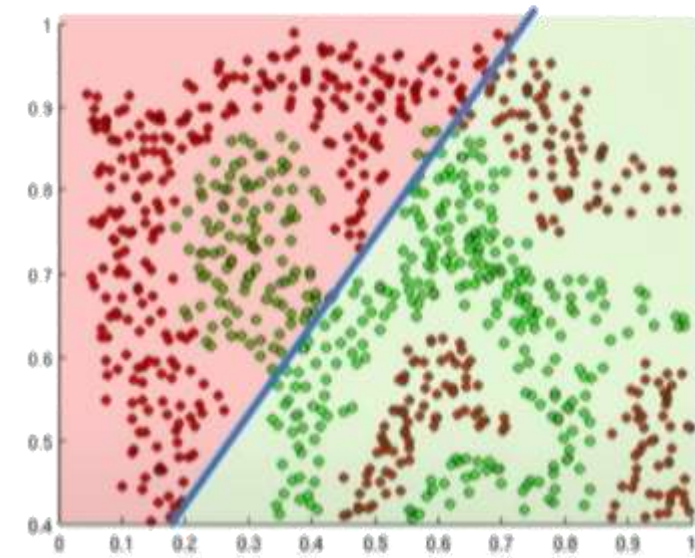
② Perceptron (感知机)



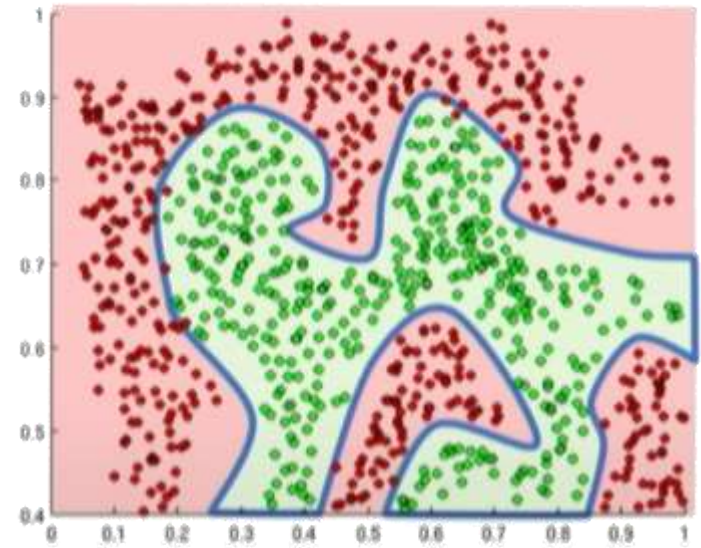
2. 机器学习

2.3 Neural Networks (NNs)

③ Activation function – Why nonlinear?



Linear activation functions produce linear decisions no matter the network size

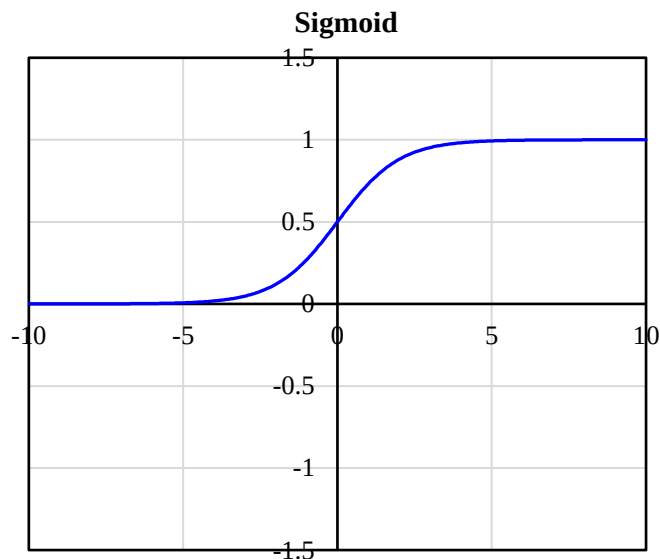


Non-linearities allow us to approximate arbitrarily complex functions

2. 机器学习

2.3 Neural Networks (NNs)

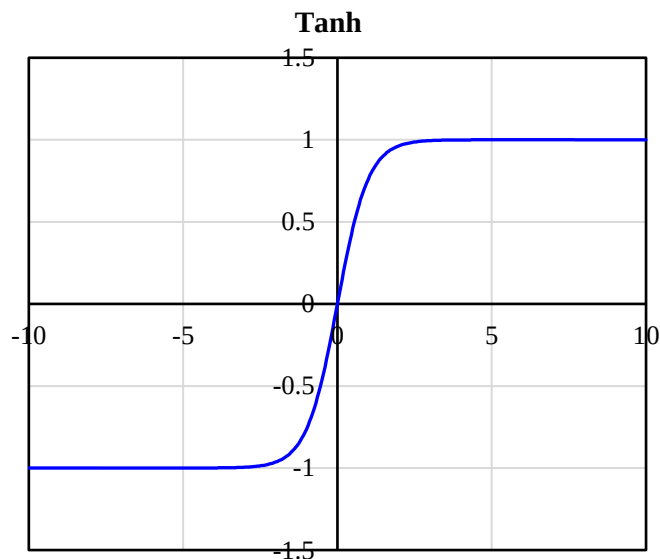
③ Activation function - Sigmoid



2. 机器学习

2.3 Neural Networks (NNs)

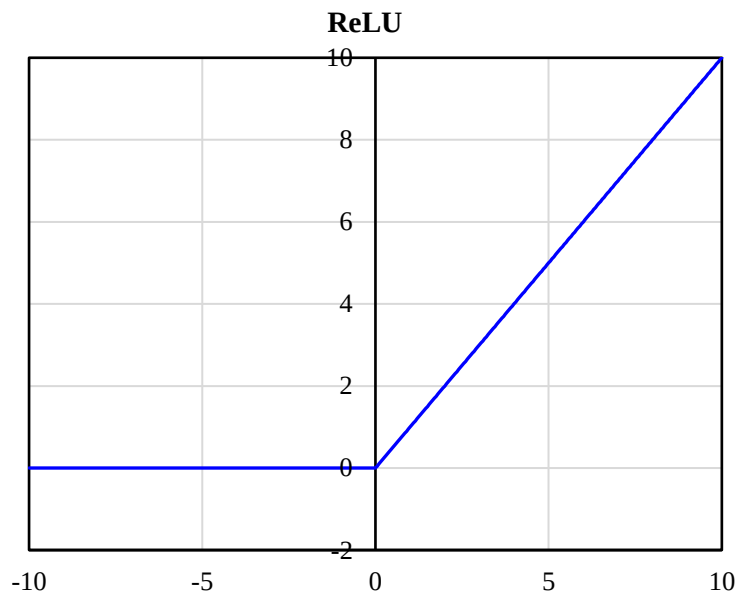
③ Activation function - Tanh



2. 机器学习

2.3 Neural Networks (NNs)

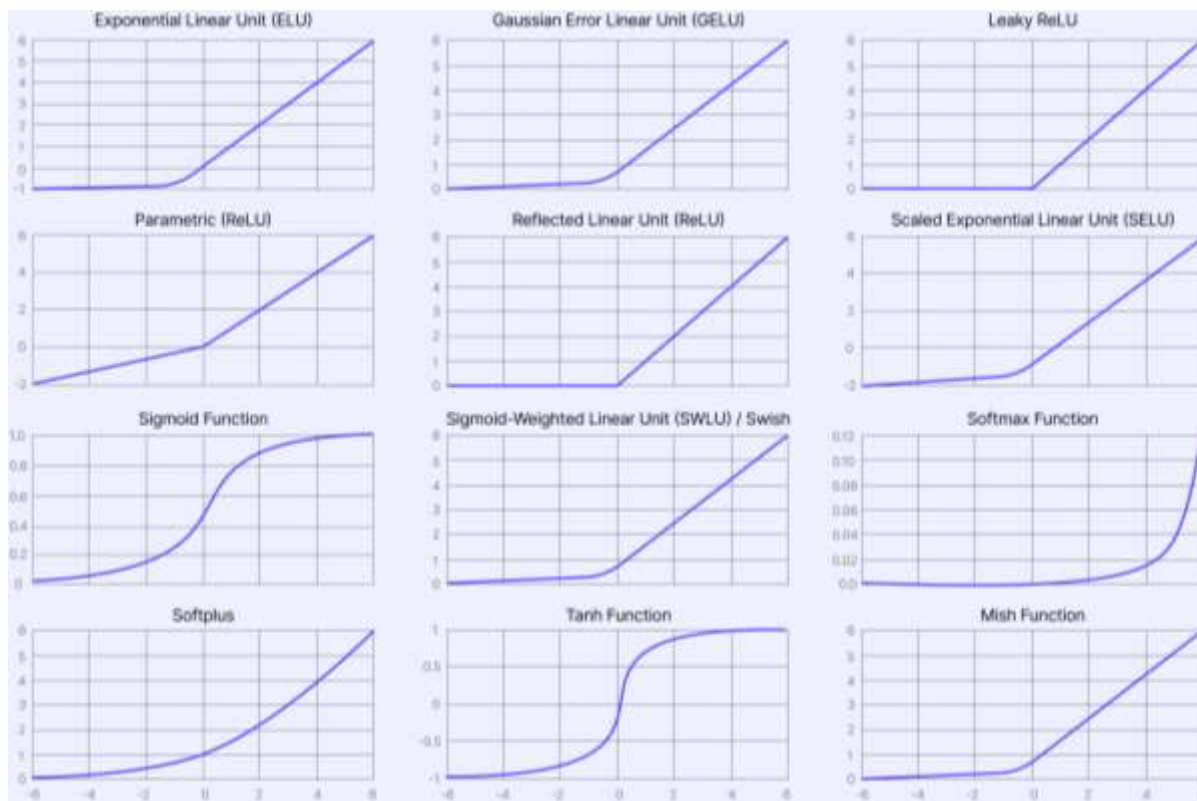
③ Activation function – ReLU (Rectified Linear Unit)



2. 机器学习

2.3 Neural Networks (NNs)

③ Activation function



2. 机器学习

2.3 Neural Networks (NNs)

④ Perceptron: Example

$$\hat{y} = g[b + (\mathbf{w}^T \mathbf{x})]$$

$$b = 1.0 \quad \mathbf{w} = \begin{bmatrix} 3 \\ -2 \end{bmatrix}$$

$$\hat{y} = g\left\{1 + [3 \quad -2] \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}\right\}$$

$$\hat{y} = g[1 + (3 * x_1 - 2 * x_2)]$$

$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} -1 \\ 2 \end{bmatrix}$$

$$\hat{y} = g[1 + (3 * x_1 - 2 * x_2)]$$

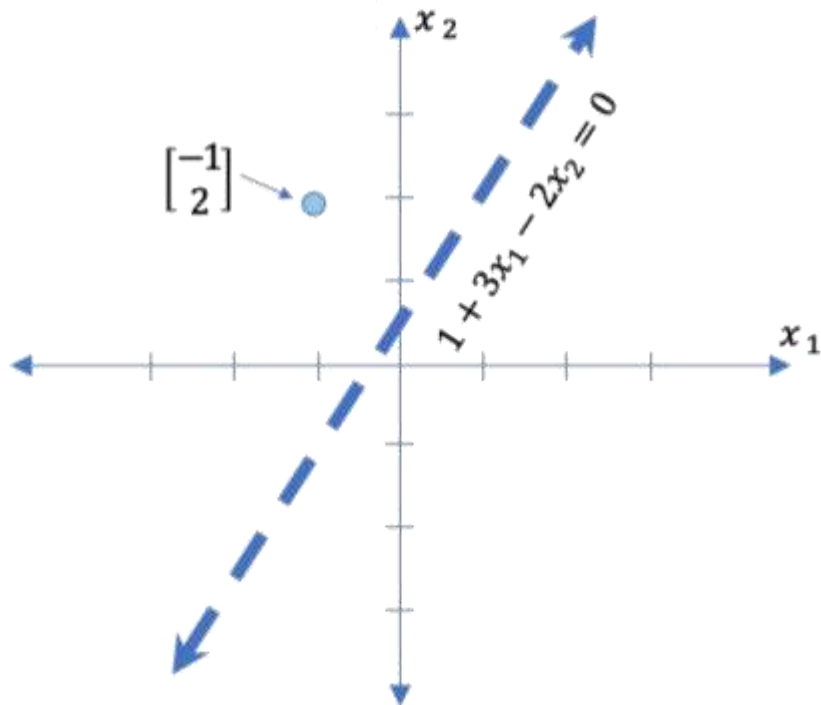
$$\hat{y} = g(-6) = \frac{1}{1 + e^6} \approx 0.002473$$

2. 机器学习

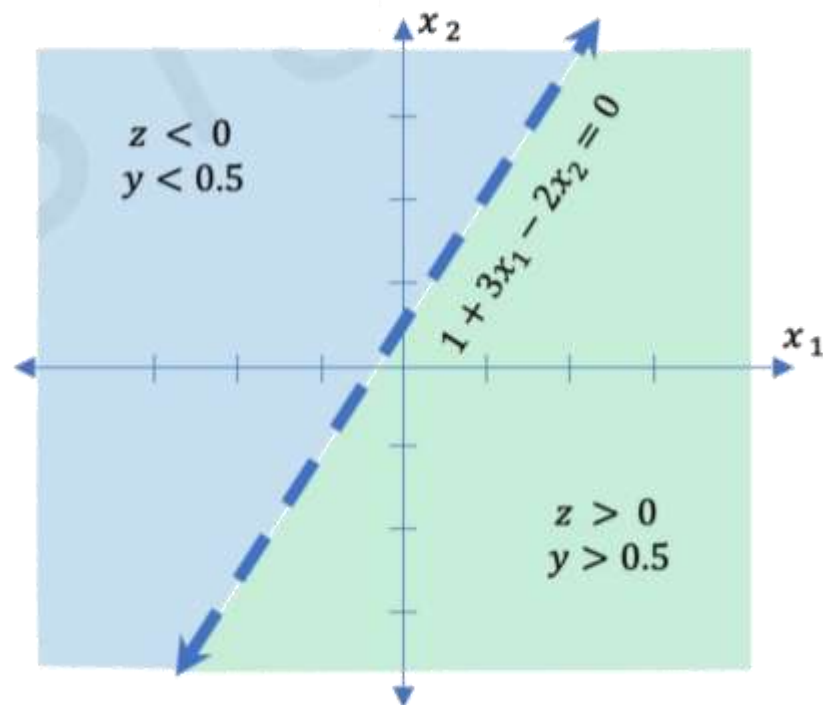
2.3 Neural Networks (NNs)

④ Perceptron: Example

$$\hat{y} = g(1 + 3x_1 - 2x_2)$$



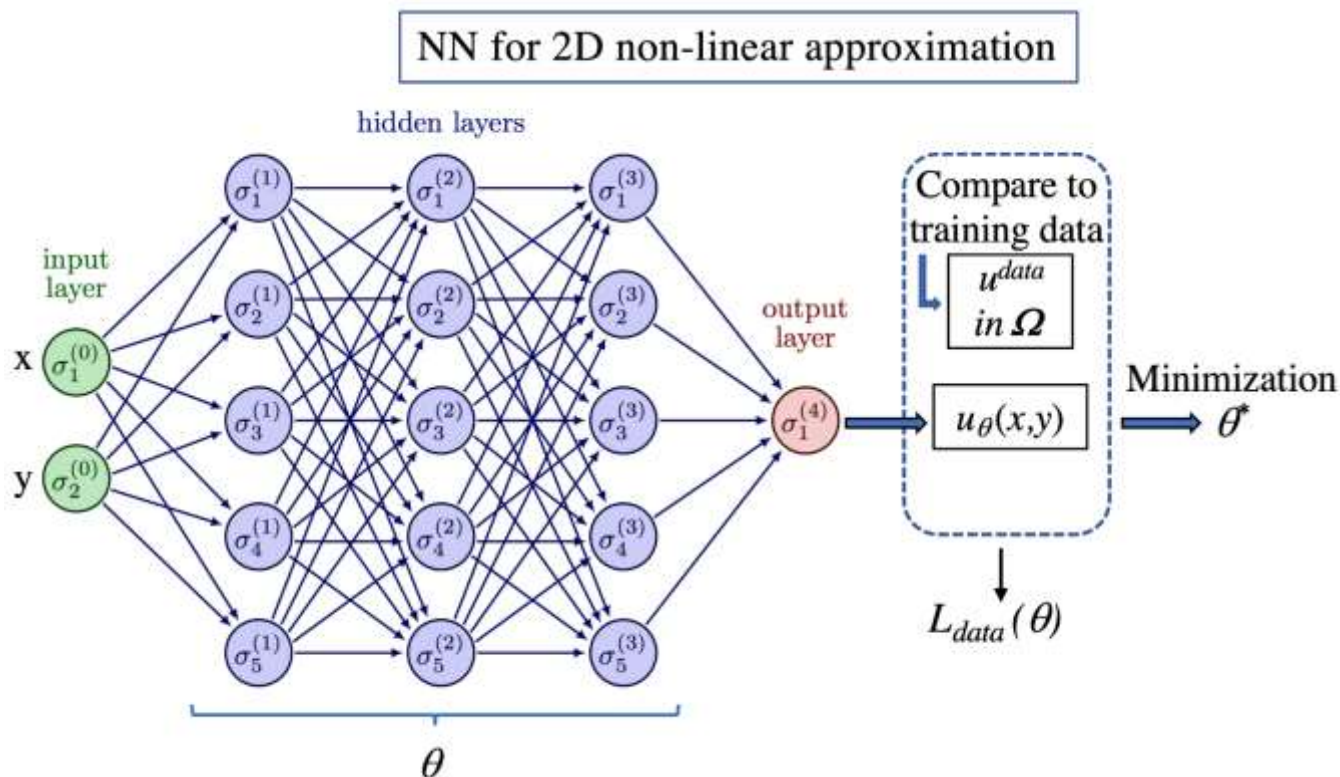
$$\hat{y} = g(1 + 3x_1 - 2x_2)$$



2. 机器学习

2.3 Neural Networks (NNs)

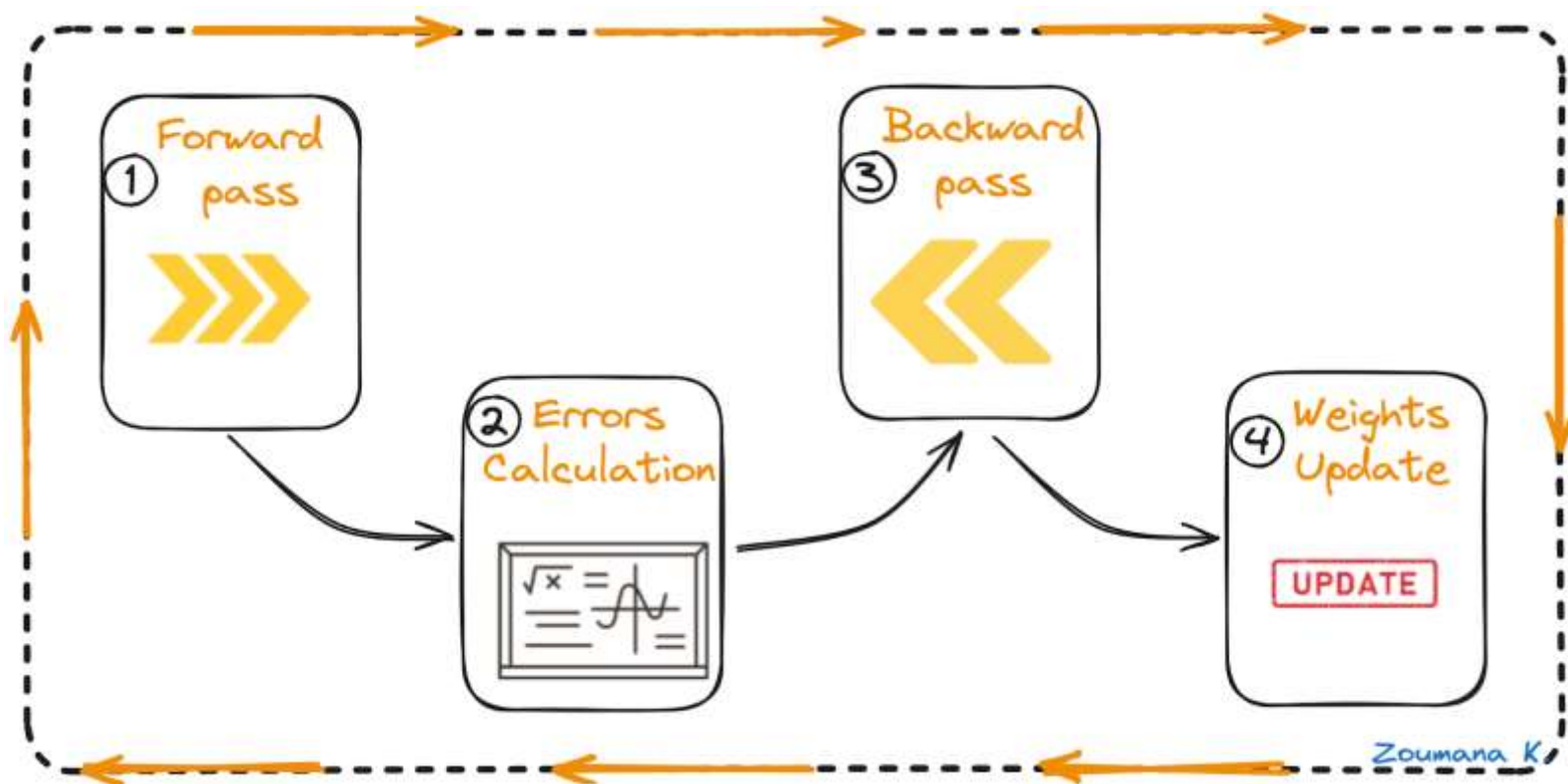
⑤ Workflow (工作原理)



2. 机器学习

2.3 Neural Networks (NNs)

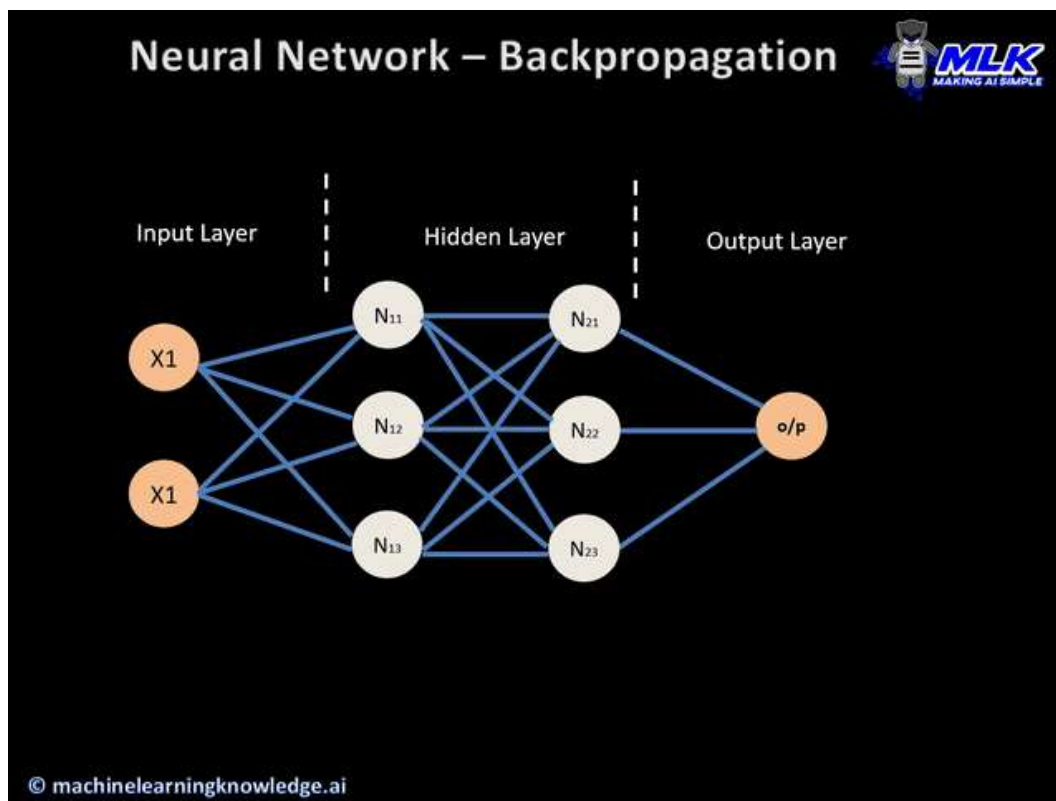
⑤ Workflow (工作原理)



2. 机器学习

2.3 Neural Networks (NNs)

⑤ Workflow (工作原理)



2. 机器学习

2.3 Neural Networks (NNs)

⑤ Workflow (工作原理)

If you have high bias (or **underfitting**)

- Increase model **complexity** (e.g., number of free parameters)
- Modify model architecture

If you have high variance (or **overfitting**)

- Increase the amount of training data
- Regularize the neural network in some other way. **Regularize** = restrict the space of possible functions a neural network can learn/**prefer** certain regions of the function space/impart a **prior** on the learning algorithm (e.g., reduce model complexity, modify model architecture, increase amount of training data, weight regularization/pruning, **additional loss terms**, **dropout**, early stopping)

2. 机器学习

2.3 Neural Networks (NNs)

⑤ Workflow (工作原理)

- How do we train neural networks (**Gradient descent**):

$$\hat{y}(x; \boldsymbol{\theta}) = \text{NN}(x; \boldsymbol{\theta})$$

$$\boldsymbol{\theta}^* = \min_{\boldsymbol{\theta}} \sum_{i=1}^N [\text{NN}(x_i; \boldsymbol{\theta}) - y_i]^2$$

$$\theta_j \leftarrow \theta_j - \gamma \frac{\partial \sum_{i=1}^N [\text{NN}(x_i; \boldsymbol{\theta}) - y_i]^2}{\partial \theta_j}$$

$$\theta_j \leftarrow \theta_j - \gamma \frac{\partial L(\boldsymbol{\theta})}{\partial \theta_j}$$

$$\frac{\partial L(\boldsymbol{\theta})}{\partial \theta_j} = \frac{\partial \sum_{i=1}^N [\text{NN}(x_i; \boldsymbol{\theta}) - y_i]^2}{\partial \theta_j} = \sum_{i=1}^N [\text{NN}(x_i; \boldsymbol{\theta}) - y_i] \frac{\partial \text{NN}(x_i; \boldsymbol{\theta})}{\partial \theta_j}$$

$$\text{NN}(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{w}_3 \{ \sigma[\mathbf{w}_2 \sigma(\mathbf{w}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2] \} + \mathbf{b}_3 = \mathbf{f} \circ \mathbf{g} \circ \mathbf{h}(\mathbf{x}; \boldsymbol{\theta})$$

where $\mathbf{h} = \mathbf{w}_1 \mathbf{x} + \mathbf{b}_1$, $\mathbf{g} = \mathbf{w}_2 \sigma(\mathbf{w}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$

2. 机器学习

2.3 Neural Networks (NNs)

⑤ Workflow (工作原理)

- How do we train neural networks (**Gradient descent**):

$$\hat{y}(x; \boldsymbol{\theta}) = \text{NN}(x; \boldsymbol{\theta})$$

$$\boldsymbol{\theta}^* = \min_{\boldsymbol{\theta}} \sum_{i=1}^N [\text{NN}(x_i; \boldsymbol{\theta}) - y_i]^2$$

$$\theta_j \leftarrow \theta_j - \gamma \frac{\partial \sum_{i=1}^N [\text{NN}(x_i; \boldsymbol{\theta}) - y_i]^2}{\partial \theta_j}$$

$$\theta_j \leftarrow \theta_j - \gamma \frac{\partial L(\boldsymbol{\theta})}{\partial \theta_j}$$

$$\frac{\partial L(\boldsymbol{\theta})}{\partial \theta_j} = \frac{\partial \sum_{i=1}^N [\text{NN}(x_i; \boldsymbol{\theta}) - y_i]^2}{\partial \theta_j} = \sum_{i=1}^N [\text{NN}(x_i; \boldsymbol{\theta}) - y_i] \frac{\partial \text{NN}(x_i; \boldsymbol{\theta})}{\partial \theta_j}$$

$$\text{NN}(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{w}_3 \{ \sigma[\mathbf{w}_2 \sigma(\mathbf{w}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2] \} + \mathbf{b}_3 = \mathbf{f} \circ \mathbf{g} \circ \mathbf{h}(\mathbf{x}; \boldsymbol{\theta})$$

where $\mathbf{h} = \mathbf{w}_1 \mathbf{x} + \mathbf{b}_1$, $\mathbf{g} = \mathbf{w}_2 \sigma(\mathbf{w}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$

2. 机器学习

2.3 Neural Networks (NNs)

⑤ Workflow (工作原理)

- How do we train neural networks (**Gradient descent**):

➤ Use the **multivariate chain rule** (= matrix multiplication of **Jacobians**)

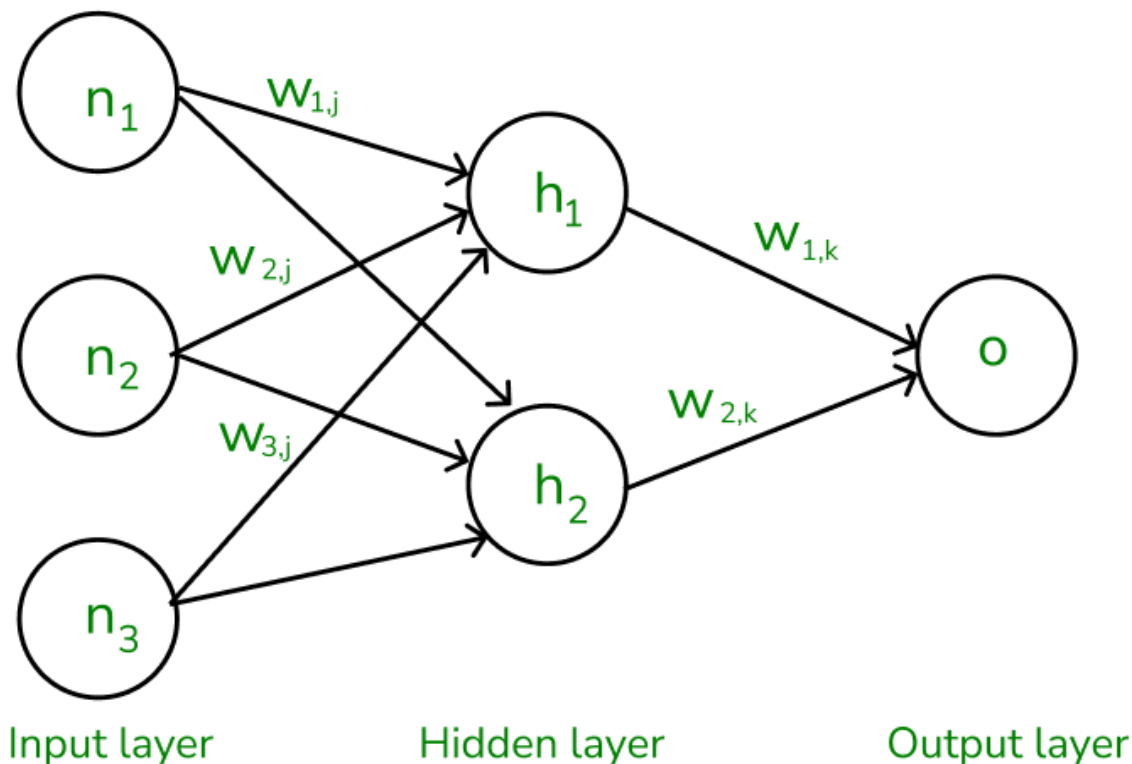
$$\frac{\partial \text{NN}}{\partial w_1} = \frac{\partial f}{\partial \mathbf{g}} \frac{\partial \mathbf{g}}{\partial \mathbf{h}} \frac{\partial \mathbf{h}}{\partial w_1}$$

$$\mathbf{J} = \frac{\partial f}{\partial \mathbf{g}} = \begin{bmatrix} \frac{\partial f_1}{\partial g_1} & \dots & \frac{\partial f_1}{\partial g_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial g_1} & \dots & \frac{\partial f_m}{\partial g_n} \end{bmatrix}$$

2. 机器学习

2.3 Neural Networks (NNs)

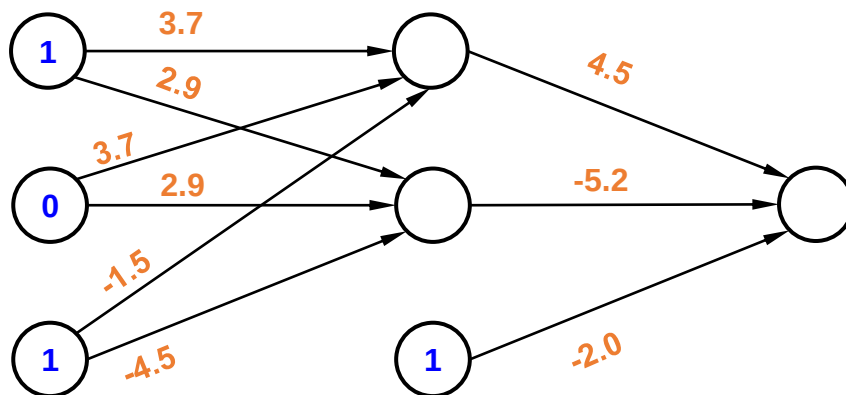
⑥ Forward pass (前馈) (Initialization)



2. 机器学习

2.3 Neural Networks (NNs)

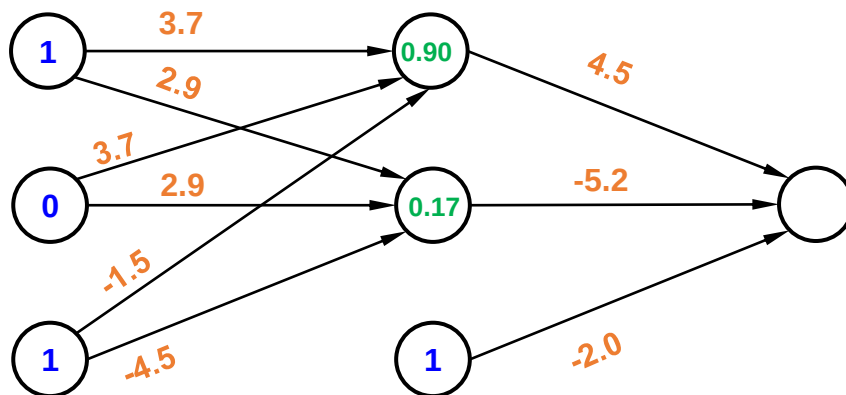
⑥ Forward pass (前馈) (Initialization)



2. 机器学习

2.3 Neural Networks (NNs)

⑥ Forward pass (前馈) (Hidden)



$$z_1 = w_{1,1}x_1 + w_{1,2}x_2 + w_{1,3}x_3 = 3.7 \times 1 + 3.7 \times 0 - 1.5 \times 1 = 2.2$$

$$g(z_1) = \frac{1}{1 + e^{-z_1}} = 0.90$$

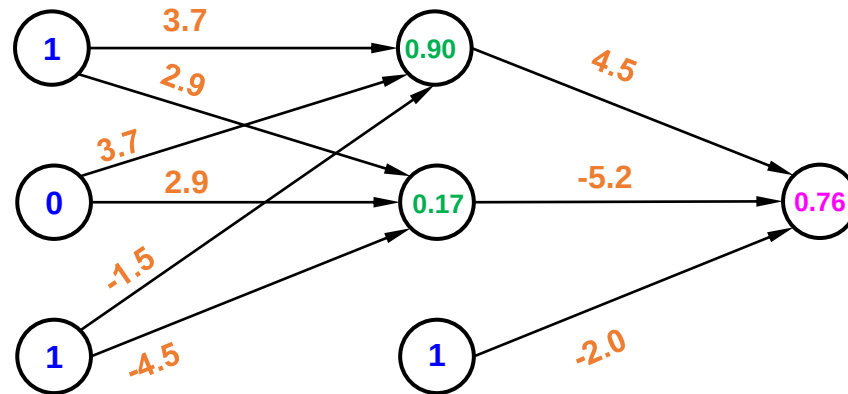
$$z_2 = w_{2,1}x_1 + w_{2,2}x_2 + w_{2,3}x_3 = 2.9 \times 1 + 2.9 \times 0 - 4.5 \times 1 = -1.6$$

$$g(z_2) = \frac{1}{1 + e^{-z_2}} = 0.17$$

2. 机器学习

2.3 Neural Networks (NNs)

⑥ Forward pass (前馈) (Output)



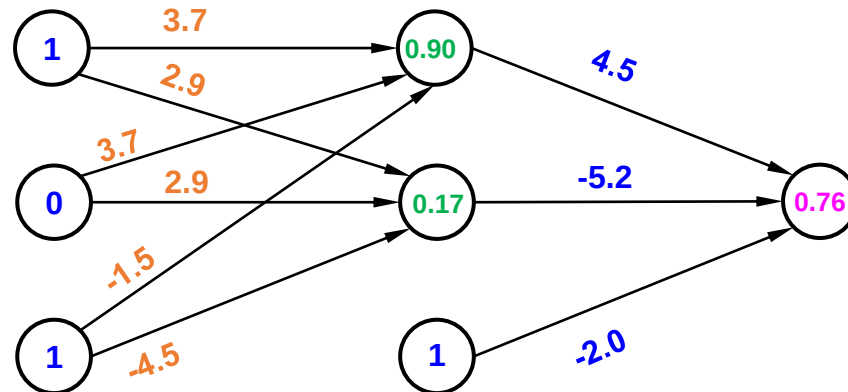
$$z_3 = w_{3,1}x_1 + w_{3,2}x_2 + w_{3,3}x_3 = 4.5 \times 0.90 - 5.2 \times 0.17 - 2.0 \times 1 = 1.17$$

$$g(z_3) = \frac{1}{1 + e^{-z_3}} = 0.76$$

2. 机器学习

2.3 Neural Networks (NNs)

⑦ Back-propagation (后向传播) training



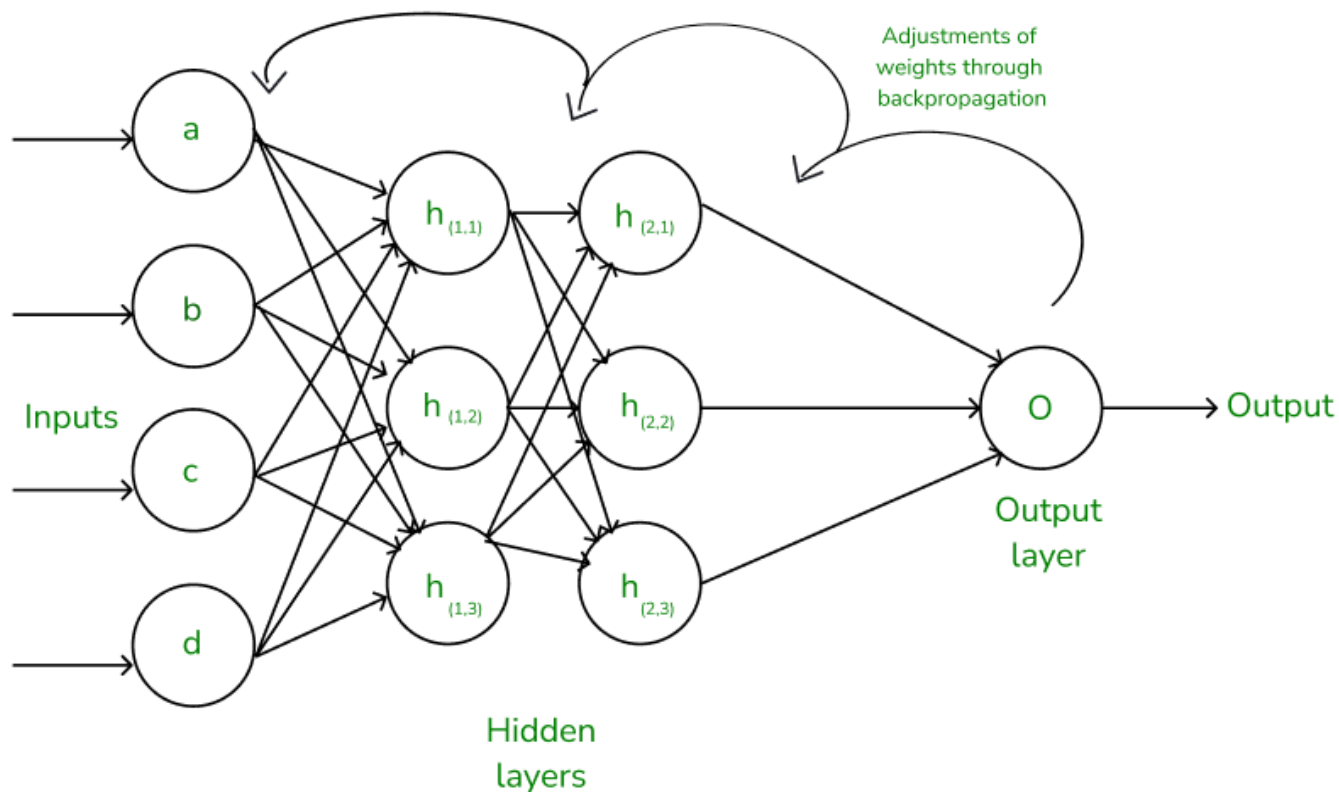
- Computed output: $y = 0.76$
- Correct output: $t = 1.0$

How do we adjust the weights w ?

2. 机器学习

2.3 Neural Networks (NNs)

⑦ Back-propagation (后向传播) training



2. 机器学习

2.3 Neural Networks (NNs)

⑦ Back-propagation (后向传播) training

Key Concepts

- **Gradient descent**

- ① Error is a function of the weights
- ② We want to reduce the error
- ③ Gradient descent: move towards the error minimum
- ④ Compute gradient → get direction to the error minimum
- ⑤ Adjust weights towards the direction lower error

- **Back-propagation**

- ① **First adjust last set of weights**
- ② **Propagate error back to each previous layer**
- ③ **Adjust their weights**

2. 机器学习

2.3 Neural Networks (NNs)

⑦ Back-propagation (后向传播) training

- Derivatives of Sigmoid

$$\frac{d}{dx} \left[\frac{f(x)}{g(x)} \right] = \frac{f'(x)g(x) - f(x)g'(x)}{g^2(x)}$$

$$\frac{d}{dx} \left(\frac{1}{1 + e^{-x}} \right) = \frac{0 \times (1 + e^{-x}) - 1 \times (-e^{-x})}{(1 + e^{-x})^2} = \frac{e^{-x}}{(1 + e^{-x})^2} = \frac{1}{1 + e^{-x}} \frac{e^{-x}}{1 + e^{-x}} = \frac{1}{1 + e^{-x}} \left(1 - \frac{1}{1 + e^{-x}} \right)$$

2. 机器学习

2.3 Neural Networks (NNs)

⑦ Back-propagation (后向传播) training

- **Final layer update**

- a. Linear combination of weights:

$$z = \sum_k w_k h_k$$

- b. Activation function:

$$y = \sigma(z)$$

- c. Error (L_2 norm):

$$\varepsilon = 0.5(t - y)^2$$

- d. Derivative of error ε with regard to one weight w_k :

$$\frac{d\varepsilon}{dw_k} = \frac{d\varepsilon}{dy} \frac{dy}{dz} \frac{dz}{dw_k}$$

- e. Derivative of error ε with respect to y :

$$\frac{d\varepsilon}{dy} = \frac{d}{dy} [0.5(t - y)^2] = -(t - y)$$

2. 机器学习

2.3 Neural Networks (NNs)

⑦ Back-propagation (后向传播) training

- **Final layer update**

- a. Linear combination of weights:

$$z = \sum_k w_k h_k$$

- b. Activation function:

$$y = \sigma(z)$$

- c. Error (L_2 norm):

$$\varepsilon = 0.5(t - y)^2$$

- d. Derivative of error ε with regard to one weight w_k :

$$\frac{d\varepsilon}{dw_k} = \frac{d\varepsilon}{dy} \frac{dy}{dz} \frac{dz}{dw_k}$$

- f. Derivative of y with respect to z :

$$\frac{dy}{dz} = y(1 - y)$$

2. 机器学习

2.3 Neural Networks (NNs)

⑦ Back-propagation (后向传播) training

- **Final layer update**

- a. Linear combination of weights:

$$z = \sum_k w_k h_k$$

- b. Activation function:

$$y = \sigma(z)$$

- c. Error (L_2 norm):

$$\varepsilon = 0.5(t - y)^2$$

- d. Derivative of error ε with regard to one weight w_k :

$$\frac{d\varepsilon}{dw_k} = \frac{d\varepsilon}{dy} \frac{dy}{dz} \frac{dz}{dw_k}$$

- g. Derivative of z with respect to w_k :

$$\frac{dz}{dw_k} = h_k$$

2. 机器学习

2.3 Neural Networks (NNs)

⑦ Back-propagation (后向传播) training

- **Final layer update**

- a. Linear combination of weights:

$$z = \sum_k w_k h_k$$

- b. Activation function:

$$y = \sigma(z)$$

- c. Error (L_2 norm):

$$\varepsilon = 0.5(t - y)^2$$

- d. Derivative of error ε with regard to one weight w_k :

$$\frac{d\varepsilon}{dw_k} = [-(t - y)][y(1 - y)]h_k$$

- h. Weight adjustment will be scaled by a fixed learning rate λ :

$$\Delta w_k = \lambda(t - y)y'h_k$$

2. 机器学习

2.3 Neural Networks (NNs)

⑦ Back-propagation (后向传播) training

- **Multiple output nodes**

- a. Error (L_2 norm):

$$\varepsilon = \sum_j 0.5(t_j - y_j)^2$$

- b. Weights $k \rightarrow j$ are adjusted according to the node:

$$\Delta w_{j \leftarrow k} = \lambda(t_j - y_j)y'_j h_k$$

2. 机器学习

2.3 Neural Networks (NNs)

⑦ Back-propagation (后向传播) training

- **Hidden layer update**

In a hidden layer, we do not have a target output value. But, we can compute how much each node contributed to downstream error.

a. Definition of error term of each node:

$$\delta_j = (t_j - y_j)y'_j$$

b. Back-propagate the error term (why this way? There is math to back it up):

$$\delta_i = \left(\sum_j w_{j \leftarrow i} \delta_j \right) y'_i$$

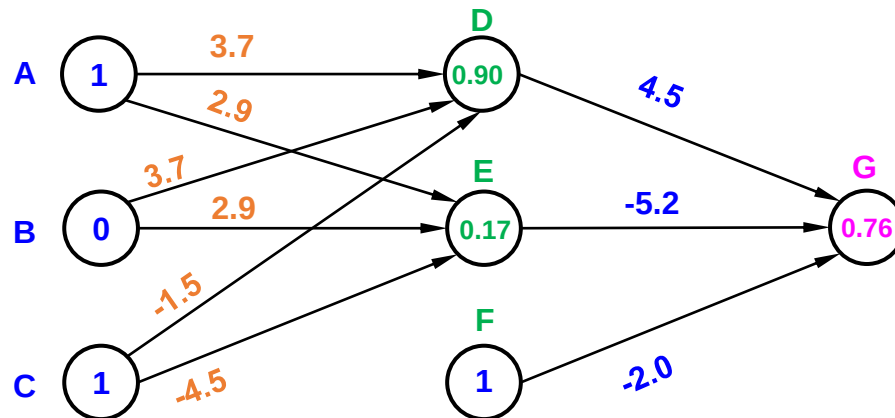
c. Universal update formula

$$\Delta w_{j \leftarrow k} = \lambda \delta_j h_k$$

2. 机器学习

2.3 Neural Networks (NNs)

⑦ Back-propagation (后向传播) training

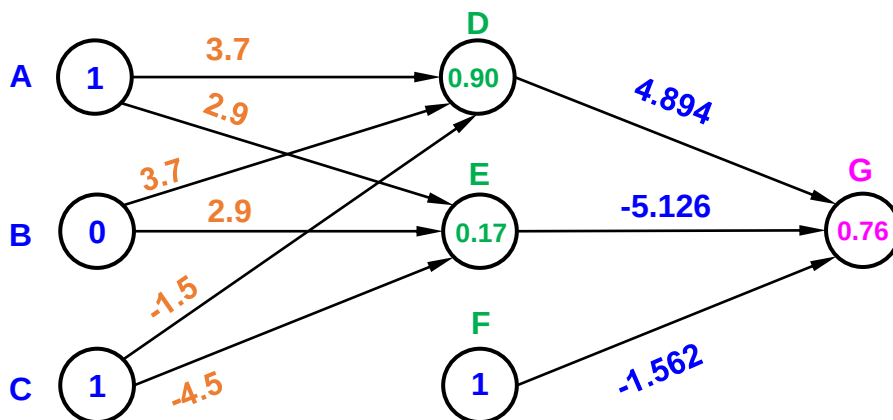


- **Computed output:** $y = 0.76$
- **Correct output:** $t = 1.0$
- **Final layer weight update (Learning rate $\lambda = 10$)**
 - $\delta_G = (t_G - y_G)y'_G = (1 - 0.76)[0.76(1 - 0.76)] = 0.0438$
 - $\Delta w_{GD} = \lambda \delta_G h_D = 10 \cdot 0.0438 \cdot 0.90 = 0.394$
 - $\Delta w_{GE} = \lambda \delta_G h_E = 10 \cdot 0.0438 \cdot 0.17 = 0.074$
 - $\Delta w_{GF} = \lambda \delta_G h_F = 10 \cdot 0.0438 \cdot 1 = 0.438$

2. 机器学习

2.3 Neural Networks (NNs)

⑦ Back-propagation (后向传播) training



- Hidden node D

$$-\delta_D = (\sum_j w_{j \leftarrow i} \delta_j) y'_D = w_{GD} \delta_G y'_D = 4.5 \cdot 0.0438 \cdot 0.09 = 0.0177$$

$$-\Delta w_{DA} = \lambda \delta_D h_A = 10 \cdot 0.0177 \cdot 1 = 0.177$$

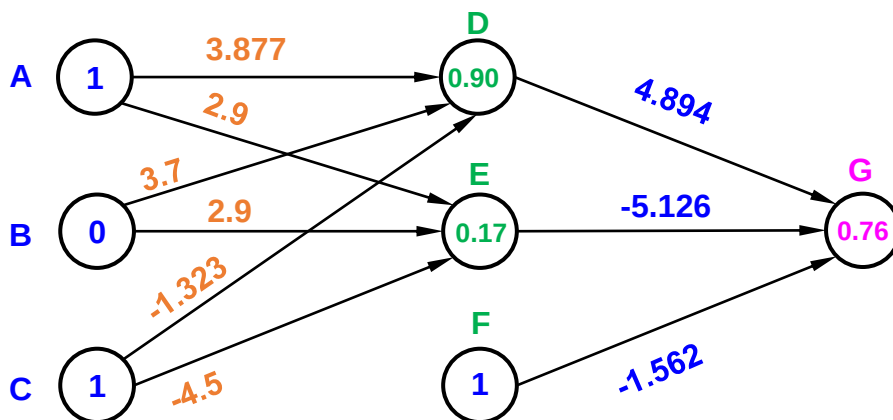
$$-\Delta w_{DB} = \lambda \delta_D h_B = 10 \cdot 0.0177 \cdot 0 = 0$$

$$-\Delta w_{DC} = \lambda \delta_D h_C = 10 \cdot 0.0177 \cdot 1 = 0.177$$

2. 机器学习

2.3 Neural Networks (NNs)

⑦ Back-propagation (后向传播) training



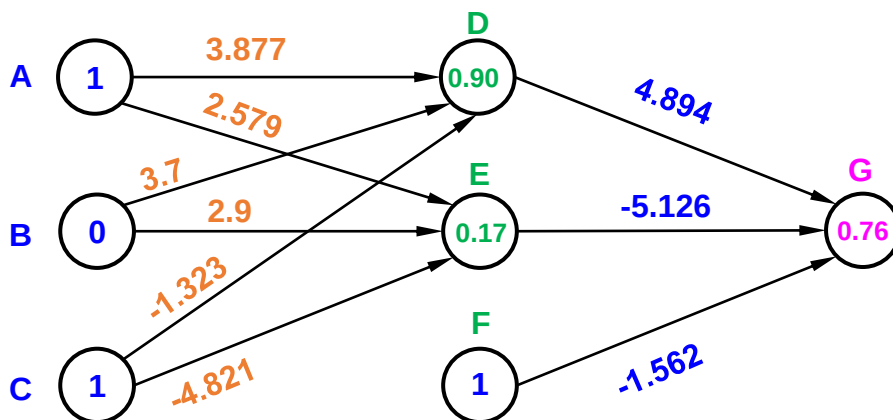
- Hidden node E

$$\begin{aligned}
 -\delta_E &= (\sum_j w_{j \leftarrow i} \delta_j) y'_E = w_{GE} \delta_G y'_E = (-5.2) \cdot 0.0438 \cdot 0.0141 = -0.0321 \\
 -\Delta w_{EA} &= \lambda \delta_E h_A = 10 \cdot (-0.0321) \cdot 1 = -0.321 \\
 -\Delta w_{EB} &= \lambda \delta_E h_B = 10 \cdot (-0.0321) \cdot 0 = 0 \\
 -\Delta w_{EC} &= \lambda \delta_E h_C = 10 \cdot (-0.0321) \cdot 1 = -0.321
 \end{aligned}$$

2. 机器学习

2.3 Neural Networks (NNs)

⑦ Back-propagation (后向传播) training



- Hidden node E

$$\begin{aligned}
 -\delta_E &= (\sum_j w_{j \leftarrow i} \delta_j) y'_E = w_{GE} \delta_G y'_E = (-5.2) \cdot 0.0438 \cdot 0.0141 = -0.0321 \\
 -\Delta w_{EA} &= \lambda \delta_E h_A = 10 \cdot (-0.0321) \cdot 1 = -0.321 \\
 -\Delta w_{EB} &= \lambda \delta_E h_B = 10 \cdot (-0.0321) \cdot 0 = 0 \\
 -\Delta w_{EC} &= \lambda \delta_E h_C = 10 \cdot (-0.0321) \cdot 1 = -0.321
 \end{aligned}$$